

SuperKart Sales Prediction

SuperKart Sales Forecasting Project

Model Deployment | Great Learning

Sarah Wolberg | August 15, 2026

Contents

3	Executive Summary	49-50	Data Background and Scope
4	Business Problem	51-61	Model Building and Benchmark
5-6	Solution Approach	62-69	Hyperparameter Tuning
7-15	Data Overview	70-72	Final Selection and Serialization
16-25	Exploratory Analysis	73-76	Deployment: Backend and Frontend
26-43	EDA Deep Dives	77-80	Insights and Recommendations
44-48	Data Preprocessing	81-82	Conclusion and Next Steps

Executive Summary

Objective. SuperKart needs product-level sales forecasts across four outlets in Tier 1 to Tier 3 cities to guide inventory and regional strategy, delivered as a deployed service rather than a one-off analysis.

Outcome. Five algorithms were built inside one preprocessing pipeline and tuned by grid search. A tuned Random Forest was selected: R-squared 0.669 on unseen data, RMSE Rs. 616, average error Rs. 487 per product-store line against a mean of Rs. 3,464. It is serialized and live behind a Flask API with a Streamlit front end on Hugging Face.

What the data says. Price drives revenue (MRP 0.79) and shelf space does not (-0.001). The estate splits into four price segments, from OUT002 at Rs. 1,763 per product to OUT003 at Rs. 4,947.

Recommendation: plan by segment, lead with price band, and pilot forecast-driven replenishment before scaling.

One caveat. The data carries no transaction dates, so this forecasts revenue level per product-store, not a time-phased quarterly plan.

Business Problem Overview

Project objective. SuperKart wants to accurately forecast the sales revenue of each product at each outlet for the upcoming quarter, and to operationalize that capability at scale. The mandate is twofold: build a predictive model on historical sales data, and deploy it as a robust forecasting service that integrates into SuperKart's decision-making systems and can be used across the store network.

The business problem. SuperKart must commit inventory and procurement a quarter ahead across four store formats and three city tiers, while planning from aggregate history. That gap drives stockouts in fast-moving categories, spoilage in perishables, working capital tied up in the wrong stock, and territory planning without a reliable benchmark.

Data and definition of success. The analysis draws on 8,763 product-store records across 12 attributes, with no missing values and no duplicates. The target is total revenue per product per store, averaging Rs. 3,464 and ranging from Rs. 33 to Rs. 8,000. Success means a model accurate enough to guide replenishment and assortment decisions, delivered as a live service planners can query rather than a notebook that stays with the data science team.

Solution Approach

Project objective: forecast quarterly sales revenue for every product at every outlet, and deploy that model as a service SuperKart's planners can query directly.

1. Data Overview and EDA

8,763 records profiled: zero missing values, zero duplicates, plus univariate and bivariate analysis.

2. Preprocessing

Store age and product category features added; encoding and scaling wrapped in one pipeline.

3. Train / Test Split

70:30 split giving 6,134 training and 2,629 test records across 10 predictors.

4. Model Building

Random Forest and Gradient Boosting trained; R-squared chosen as the primary metric.

5. Tuning and Selection

3-fold GridSearchCV on both models; the tuned Random Forest won and was serialized.

6. Deployment

Flask API in a Docker Space plus a Streamlit front end, both public on Hugging Face.

Solution Approach Explained

From data to features. Exploratory analysis confirmed clean data with no missing or duplicate rows, so the effort went into features: store age derived from the establishment year, a coarse product family read from the product ID prefix, and consolidated product categories. Identifiers were dropped and high-value sales kept as genuine, leaving 10 predictors.

Why a single pipeline. One-hot encoding for categorical fields and scaling for numeric ones are bundled with the estimator in one scikit-learn pipeline, so the exact transformations used in training are reproduced automatically at prediction time and cannot drift in production.

Model and metric choice. Tree-based ensembles suit this data because the drivers are non-linear and mix categorical with numeric fields. R-squared is the primary metric, with RMSE, MAE and MAPE reported in rupee terms so planners can judge tolerable error. The data was split 70:30 with a fixed random seed.

Tuning, selection and deployment. Three-fold grid search tuned tree depth, estimator count and feature sampling. The tuned Random Forest was selected on test R-squared of 0.669, RMSE of Rs. 616 and MAPE of 18.7%, with training and test scores close enough to rule out material overfitting. It is serialized with joblib, served by a Flask API in a Docker Space, and fronted by a Streamlit app.

Data Overview: Source and Description

A single flat file, SuperKart.csv, holds 8,763 product-store sales records. No external data was used, so every conclusion traces back to this one source.

8,763

records (rows)

12

attributes (columns)

0

missing values

0

duplicate rows

Column types: 4 float (weight, display area, MRP, revenue), 1 integer (establishment year), 7 text (identifiers and categorical attributes).

Target variable: Product_Store_Sales_Total - mean Rs. 3,464, median Rs. 3,452, range Rs. 33 to Rs. 8,000.

So what: clean product-by-store history, enough to train a forecast with no data repair.

Understand the shape of the dataset

```
print(f"There are {data.shape[0]} rows and {data.shape[1]} columns.") #Complete the code to display the number of columns
```

There are 8763 rows and 12 columns.

Data Dictionary: What Each Attribute Captures

PRODUCT ATTRIBUTES

Product_Id - unique product code: two letters then a number

Product_Weight - weight of the product, 4.0 to 22.0

Product_Sugar_Content - low sugar, regular or no sugar

Product_Allocated_Area - share of the store's display area given to the product

Product_Type - 16 broad categories, from dairy to household

Product_MRP - maximum retail price, Rs. 31 to Rs. 266

STORE ATTRIBUTES AND TARGET

Store_Id - unique outlet code; four outlets appear in the data

Store_Establishment_Year - year the outlet opened, 1987 to 2009

Store_Size - high, medium or small by square footage

Store_Location_City_Type - Tier 1, 2 or 3 by standard of living

Store_Type - departmental store, supermarket type 1 or 2, food mart

Product_Store_Sales_Total - TARGET: revenue of that product in that store

Column Types and Structure: All 12 Fields

Check the data types of the columns for the dataset

```
! data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8763 entries, 0 to 8762
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                ---
0   Product_Id                            8763 non-null   object
1   Product_Weight                        8763 non-null   float64
2   Product_Sugar_Content                8763 non-null   object
3   Product_Allocated_Area               8763 non-null   float64
4   Product_Type                         8763 non-null   object
5   Product_MRP                         8763 non-null   float64
6   Store_Id                             8763 non-null   object
7   Store_Establishment_Year             8763 non-null   int64
8   Store_Size                           8763 non-null   object
9   Store_Location_City_Type             8763 non-null   object
10  Store_Type                           8763 non-null   object
11  Product_Store_Sales_Total            8763 non-null   float64
dtypes: float64(4), int64(1), object(7)
memory usage: 821.7+ KB
```

Read: 8,763 non-null on every column; types split 4 float, 1 integer, 7 text.

- **Nothing missing.** All 12 columns report 8,763 non-null entries.
- **Four numeric measures.** Weight, display area, MRP and revenue feed the model directly.
- **An integer that is a label.** Establishment year has four values; used as store age.
- **Seven text fields.** All categorical and all need encoding; Product_Id was dropped.
- **Small footprint.** 822 KB in memory, so retraining is cheap to run on a schedule.
- **Business impact.** No cleaning backlog, so encoding is the only mandatory prep step.

Sample Records: First and Last Rows

The head and tail confirm the grain of the data. Seven of the twelve columns are shown below: the first two rows of the file and the last two.

Product_Id	Weight	Sugar	MRP	Product_Type	Store	Sales (Rs.)
FD6114	12.66	Low Sugar	117.08	Frozen Foods	OUT004	2,842.40
FD7839	16.54	Low Sugar	171.43	Dairy	OUT003	4,830.02
NC2471	13.48	No Sugar	88.58	Health and Hygiene	OUT001	2,443.42
FD3061	4.73	Low Sugar	224.93	Snack Foods	OUT002	2,186.08

Observation: every row carries a unique Product_Id, so the file holds one record per product per store with no repeats over time.

Business read: the ID prefix encodes the product family - FD food, DR drinks, NC non-consumable - which is usable signal even though the ID itself is not.

Notebook Output: head() and tail()

View the first and last 5 rows of the dataset

```
[6]: data.head() #Complete the code to display the first 5 rows of the dataset
```

	Product_Id	Product_Weight	Product_Sugar_Content	Product_Allocated_Area	Product_Type	Product_MRP	Store_Id	Store_Establishment_Year	Store_Size	Store_Location_City_Type
0	FD6114	12.66	Low Sugar	0.027	Frozen Foods	117.08	OUT004	2009	Medium	Tier 2
1	FD7839	16.54	Low Sugar	0.144	Dairy	171.43	OUT003	1999	Medium	Tier 1
2	FD5075	14.28	Regular	0.031	Canned	162.08	OUT001	1987	High	Tier 2
3	FD8233	12.10	Low Sugar	0.112	Baking Goods	186.31	OUT001	1987	High	Tier 2
4	NC1180	9.57	No Sugar	0.010	Health and Hygiene	123.67	OUT002	1998	Small	Tier 3

```
[7]: data.tail() #Complete the code to display the last 5 rows of the dataset
```

	Product_Id	Product_Weight	Product_Sugar_Content	Product_Allocated_Area	Product_Type	Product_MRP	Store_Id	Store_Establishment_Year	Store_Size	Store_Location_City_Type
8758	NC7546	14.80	No Sugar	0.016	Health and Hygiene	140.53	OUT004	2009	Medium	Tier
8759	NC584	14.06	No Sugar	0.142	Household	144.51	OUT004	2009	Medium	Tier
8760	NC2471	13.48	No Sugar	0.017	Health and Hygiene	88.58	OUT001	1987	High	Tier
8761	NC7187	13.89	No Sugar	0.193	Household	168.44	OUT001	1987	High	Tier
8762	FD306	14.73	Low Sugar	0.177	Snack Foods	224.93	OUT002	1998	Small	Tier

So what: all 12 columns are populated end to end, and the file is ordered by product family rather than by date, so there is no time dimension to forecast along.

Statistical Summary of the Data

All five numeric attributes across 8,763 records. Mean and median sit close together on weight, price and revenue, so the distributions are broadly symmetric with no heavy skew to correct.

Attribute	Mean	Std dev	Min	Median	Max
Product_Weight	12.65	2.22	4.00	12.66	22.00
Product_Allocated_Area	0.069	0.048	0.004	0.056	0.298
Product_MRP (Rs.)	147.03	30.69	31.00	146.74	266.00
Store_Establishment_Year	2002	8.39	1987	2009	2009
Product_Store_Sales_Total (Rs.)	3,464.00	1,065.63	33.00	3,452.34	8,000.00

Read: revenue spans Rs. 33 to Rs. 8,000 around a mean of Rs. 3,464.

Caution: establishment year has only four values, so it acts as a store label.

Notebook Output: Statistical Summary

Statistical summary of the data

Let's check the statistical summary of the data.

```
0]: data.describe(include="all").T
```

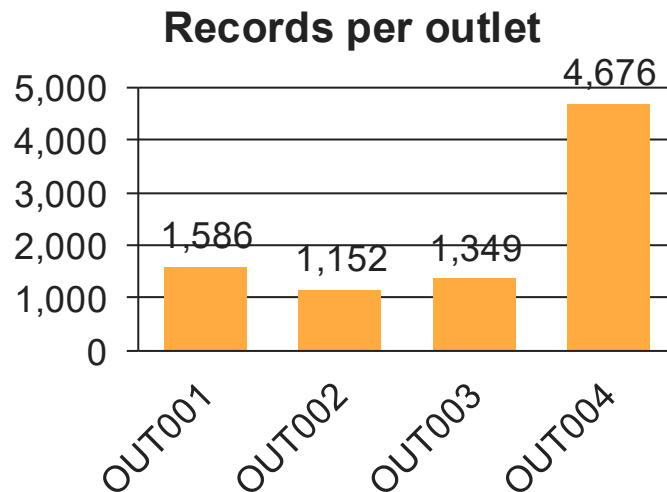
	count	unique	top	freq	mean	std	min	25%	50%	75%	max
Product_Id	8763	8763	FD6114	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product_Weight	8763.0	NaN	NaN	NaN	12.653792	2.21732	4.0	11.15	12.66	14.18	22.0
Product_Sugar_Content	8763	4	Low Sugar	4885	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product_Allocated_Area	8763.0	NaN	NaN	NaN	0.068786	0.048204	0.004	0.031	0.056	0.096	0.298
Product_Type	8763	16	Fruits and Vegetables	1249	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product_MRP	8763.0	NaN	NaN	NaN	147.032539	30.69411	31.0	126.16	146.74	167.585	266.0
Store_Id	8763	4	OUT004	4676	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Store_Establishment_Year	8763.0	NaN	NaN	NaN	2002.032751	8.388381	1987.0	1998.0	2009.0	2009.0	2009.0
Store_Size	8763	3	Medium	6025	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Store_Location_City_Type	8763	3	Tier 2	6262	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Store_Type	8763	4	Supermarket Type2	4676	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product_Store_Sales_Total	8763.0	NaN	NaN	NaN	3464.00364	1065.630494	33.0	2761.715	3452.34	4145.165	8000.0

Categoricals: 4 sugar labels, 16 product types, 4 stores, 3 sizes, 3 city tiers, 4 store types.

Business impact: assortment and estate are both concentrated, so one national plan misfits both ends.

Data Quality and Label Issues

- **Clean on the basics.** No duplicate rows and no missing values across all 12 columns.
- **Inconsistent labels.** "reg" (108 rows) duplicates "Regular" (2,251) and must be merged.
- **Two meanings in one field.** All 1,519 non-consumables are tagged "No Sugar" by default.
- **Identifiers carry no signal.** Product_Id is unique on all 8,763 rows; only its prefix was kept.
- **Store fields move together.** Four outlets, each with a fixed size, tier, type and year.
- **Sample is concentrated.** OUT004 supplies 53% of records, skewing error toward one outlet.



Implication: accuracy should be tracked per outlet, not just overall.

Duplicate and Missing Value Checks

Checking for duplicate values

```
# checking for duplicate values  
data.duplicated().sum()
```

```
0
```

Checking for missing values

```
# checking for missing values in the data  
data.isnull().sum() #Complete the code to compute the number of missing values.
```

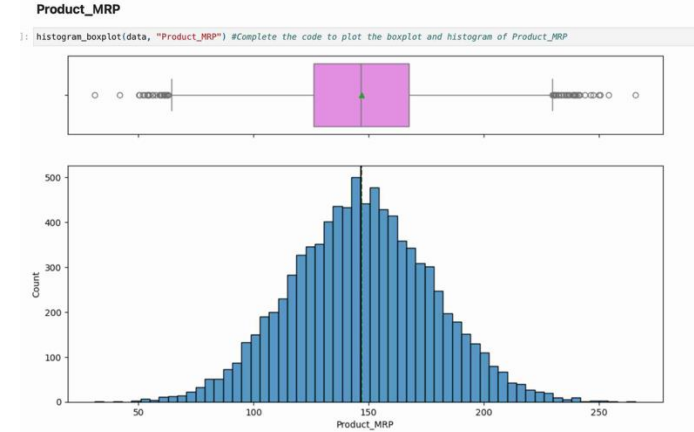
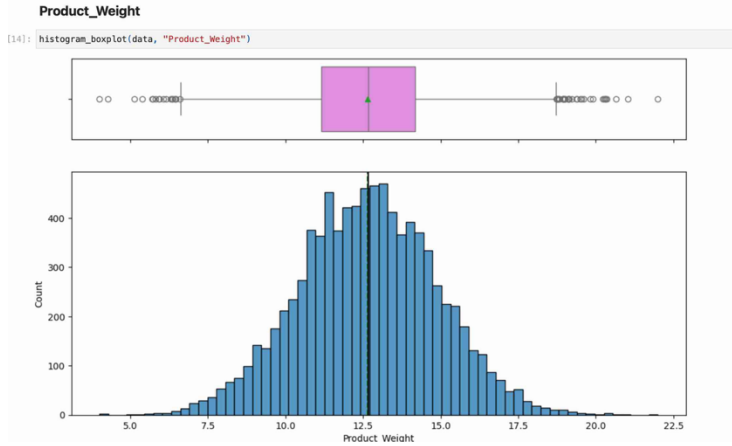
```
Product_Id           0  
Product_Weight       0  
Product_Sugar_Content 0  
Product_Allocated_Area 0  
Product_Type         0  
Product_MRP          0  
Store_Id             0  
Store_Establishment_Year 0  
Store_Size           0  
Store_Location_City_Type 0  
Store_Type           0  
Product_Store_Sales_Total 0  
dtype: int64
```

- **Zero duplicate rows.** No product-store record is repeated, so no revenue is double counted in training.
- **Zero missing values.** All 12 columns are complete, so no imputation was applied and no rows were discarded.
- **Why it matters.** Imputation and de-duplication are the two steps that most often bias a retail forecast. Neither was needed here, so measured accuracy reflects the data rather than a cleaning choice.
- **Watch instead for label quality.** Completeness is not correctness: the "reg" versus "Regular" split on sugar content is the one real defect found, and it was merged before encoding.

EDA: Approach and Key Results

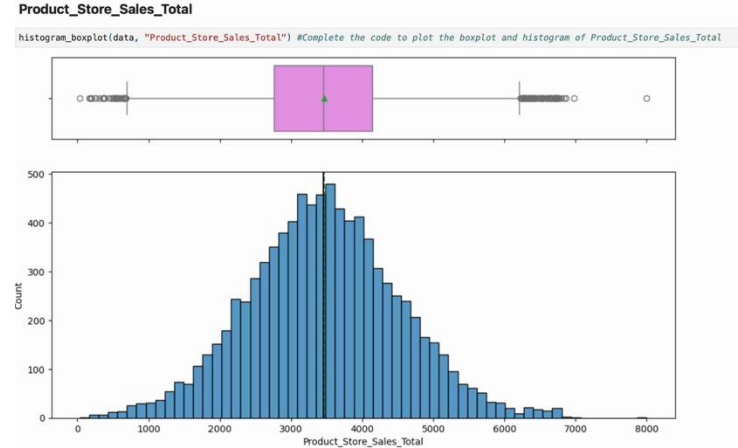
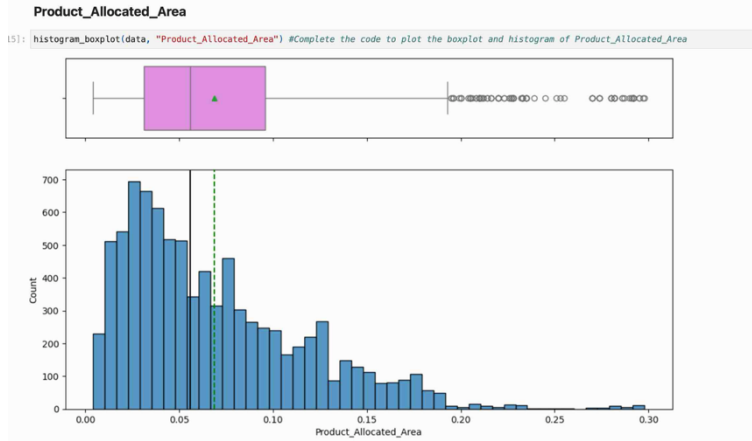
- **Method.** Univariate distributions and boxplots for the four numeric fields, labelled bar charts for the categorical fields, then bivariate tests of every field against revenue.
- **Price and weight drive revenue.** Product MRP correlates 0.79 with revenue and product weight 0.74. These two fields carry almost all of the predictive signal.
- **Display area does not.** Allocated display area correlates -0.001 with revenue: shelf space as currently assigned has no measurable effect on sales.
- **Store format sets the level.** Average revenue per product runs from Rs. 4,947 at the Tier-1 departmental store to Rs. 1,763 at the Tier-3 food mart, a 2.8x spread.
- **Product attributes barely matter.** Sugar content (Rs. 3,444 to Rs. 3,502) and product family (Rs. 3,437 to Rs. 3,469) are flat against revenue.
- **Distributions are workable.** Weight, MRP and revenue are near-symmetric; only display area is right-skewed. Outliers are genuine high-value sales and were kept.

Univariate: Product Weight and Price



- **Product_Weight.** Near-symmetric: mean 12.65, median 12.66, range 4.0 to 22.0. Boxplot tails are genuine light and heavy lines, so none were removed.
- **Product_MRP.** Also near-symmetric: mean Rs. 147.03, median Rs. 146.74, range Rs. 31 to Rs. 266, concentrated in a Rs. 126 to 168 band.
- **Business read.** Clean, skew-free fields that predict well; the tight MRP band shows a mid-price-heavy assortment.

Univariate: Display Area and Revenue

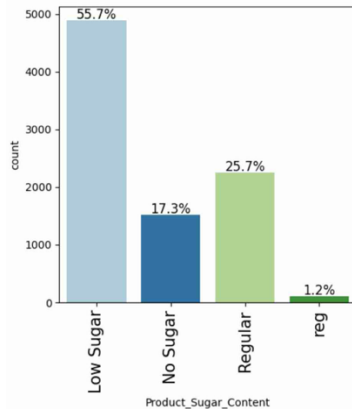


- **Product_Allocated_Area.** The only right-skewed field: mean 0.069 above a median of 0.056, with a long tail to 0.298. Most products get a small slice of shelf; a few get a lot.
- **Product_Store_Sales_Total.** The target is near-symmetric: mean Rs. 3,464, median Rs. 3,452, range Rs. 33 to Rs. 8,000, with the bulk between Rs. 2,762 and Rs. 4,145.
- **Business read.** Shelf space is allocated unevenly but, as the bivariate view shows, that unevenness buys no extra revenue. The symmetric target means a single model can serve the whole range without transformation.

Univariate: Sugar Content and Product Type

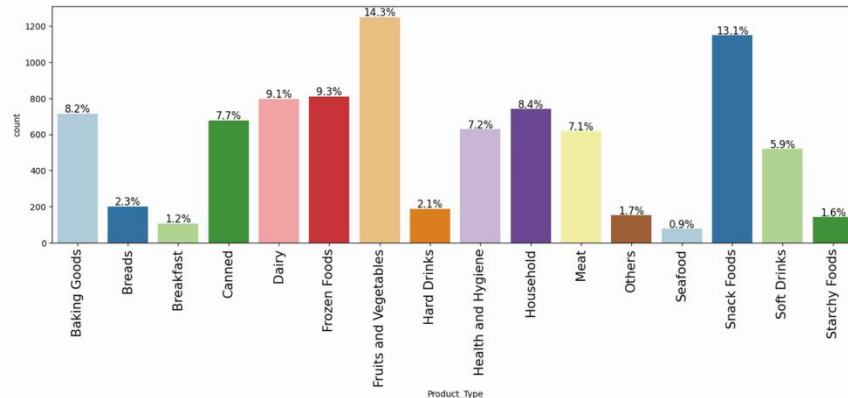
Product_Sugar_Content

```
labeled_barplot(data, "Product_Sugar_Content", perc=True)
```



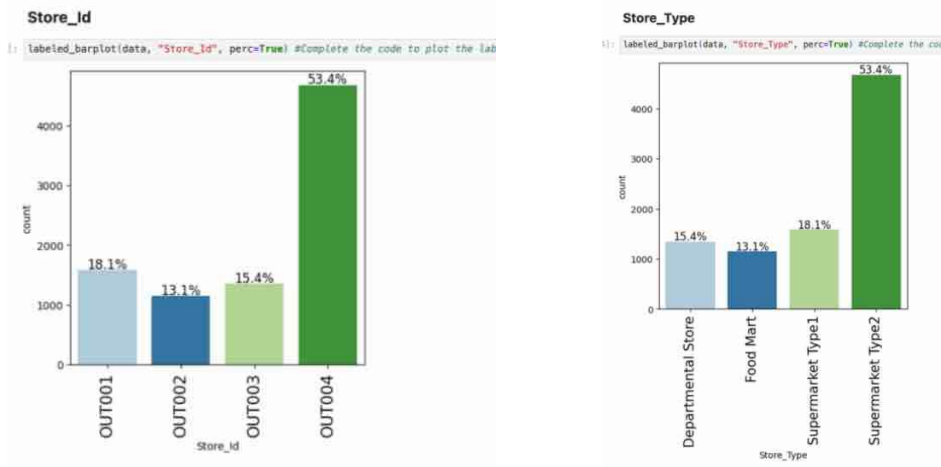
Product_Type

```
labeled_barplot(data, "Product_Type", perc=True) #Complete the code to plot the labeled barplot of Product_Type with the percentages being displayed
```



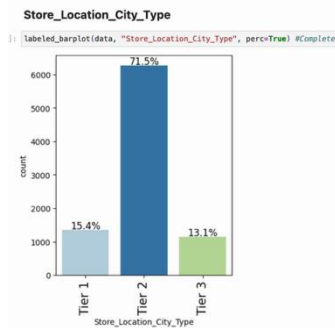
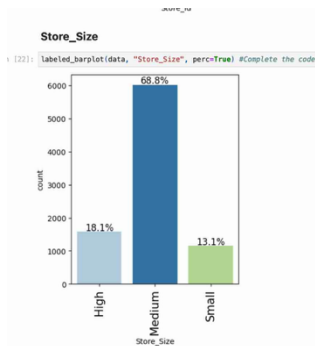
- **Sugar content.** Low Sugar 55.7%, Regular 25.7%, No Sugar 17.3%, plus a stray "reg" at 1.2% - the same category as Regular, merged before modelling.
- **Product type.** 16 categories led by Fruits and Vegetables 14.3% and Snack Foods 13.1%; Seafood 0.9%, Breakfast 1.2% and Starchy Foods 1.6% are marginal.
- **Business read.** Over half the assortment is low-sugar, and the top two fresh and impulse categories carry a quarter of all lines - the categories whose availability matters most to revenue.

Univariate: Outlets and Store Formats



- **Four outlets, one dominant.** OUT004 supplies 53.4% of records; OUT001 18.1%, OUT003 15.4%, OUT002 13.1%. Formats mirror outlets one-for-one.
- **Issue to flag.** Each format maps to exactly one outlet, so a format effect cannot be separated from a store effect and predictions will not transfer to a new store.
- **So what.** Half the training signal is one Tier-2 supermarket: track accuracy per outlet and pilot beyond OUT004.

Univariate: Store Size and City Tier



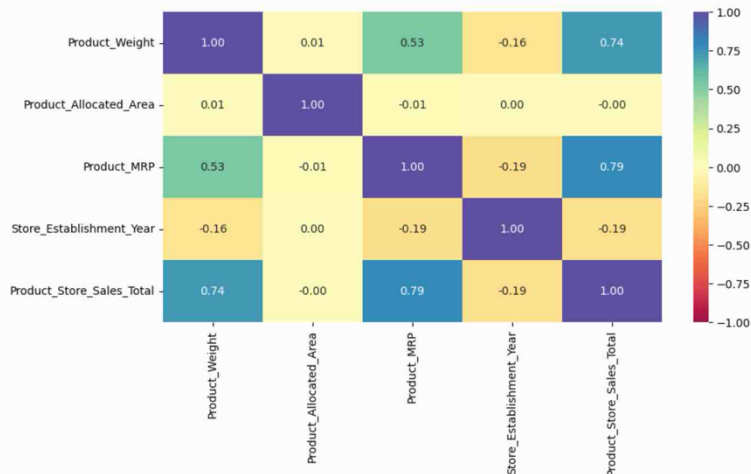
- **Store size.** Medium 68.8%, High 18.1%, Small 13.1% - the estate in this data is mostly medium-format.
- **City tier.** Tier 2 71.5%, Tier 1 15.4%, Tier 3 13.1%, so nearly three quarters of records come from mid-tier cities.
- **So what.** Any national rule learned from this data is effectively a medium-store, Tier-2 rule. Tier 1 and Tier 3 are thin, and they are exactly the two ends where revenue per product diverges most (Rs. 4,947 versus Rs. 1,763).
- **Recommendation.** Treat forecasts for Tier-1 and Tier-3 outlets as lower-confidence and validate them against actuals before letting them drive purchase orders.

Bivariate: Correlation Matrix

Bivariate Analysis

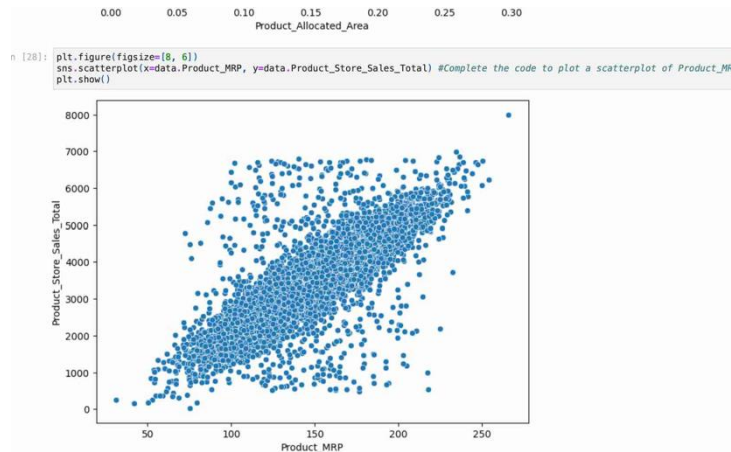
Correlation matrix

```
5]: cols_list = data.select_dtypes(include=np.number).columns.tolist()
plt.figure(figsize=(10, 5))
sns.heatmap(
    data[cols_list].corr(), annot=True, vmin=-1, vmax=1, fmt=".2f", cmap="Spectral"
)
plt.show()
```

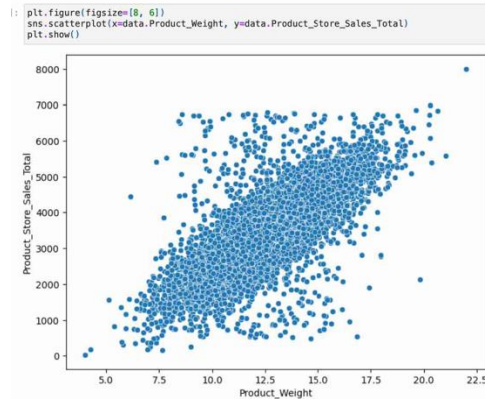


- **Price leads.** MRP correlates 0.79 with revenue, the strongest driver in the data.
- **Weight follows.** 0.74 with revenue, but 0.53 with MRP, so the two overlap.
- **Display area is inert.** -0.00 against revenue: shelf allocation is unrelated to sales.
- **Newer store, lower unit revenue.** Establishment year -0.19; the 2009 outlet is high-volume, low-value.
- **So what.** Price is the clear lever and shelf space is not; overlapping drivers favour a tree-based model.

Bivariate: Price and Weight vs Revenue

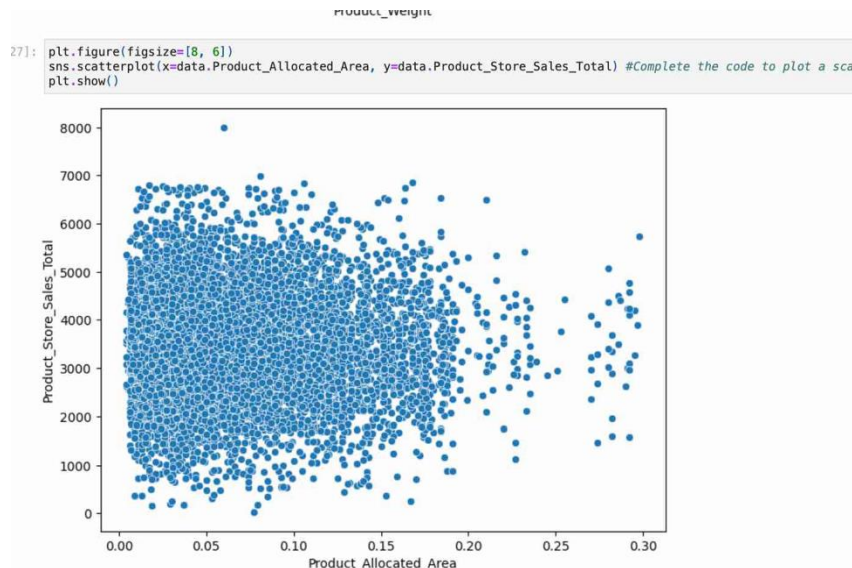


Let's check the distribution of our target variable i.e Product_Store_Sales_Total with the numeric columns



- **MRP vs revenue (left).** A clear upward band: higher-priced products earn more per store. The spread widens above Rs. 150, so price sets the level but not the certainty.
- **Weight vs revenue (right).** The same upward pattern, slightly more diffuse, consistent with weight partly proxying for price.
- **So what.** Revenue is driven by what a product costs, not how much shelf it gets. Assortment and pricing decisions move sales; space reshuffling does not.

Bivariate: Display Area vs Revenue

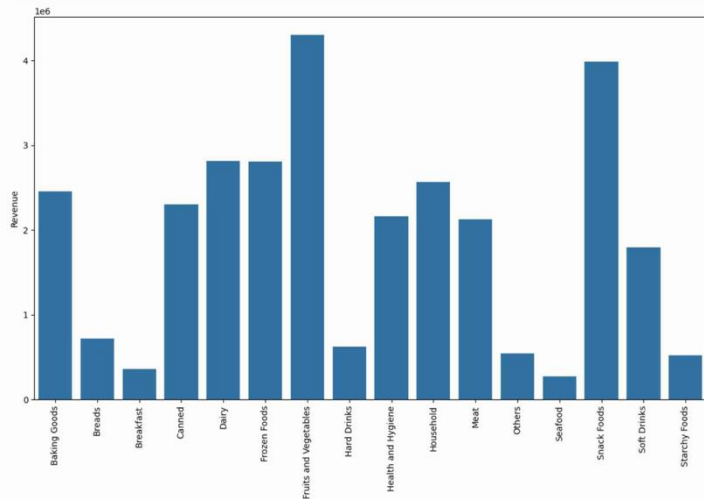


- **What the plot shows.** A flat cloud. At every level of allocated display area, from 0.01 to 0.30, revenue spans the same Rs. 0 to Rs. 7,000 range. There is no upward drift.
- **The number behind it.** Correlation of -0.001, the weakest relationship of any field tested against revenue.
- **Issue.** Shelf space is currently assigned in a way that carries no revenue signal, and the widest allocations sit on products that sell no better than the narrowest.
- **Recommendation.** Stop treating display area as a growth lever on its own. Reassign space by category sales velocity and prove it with a controlled in-store test before rolling out a new planogram rule.

Bivariate: Revenue by Product Category

Let us see from which product type the company is generating most of the revenue

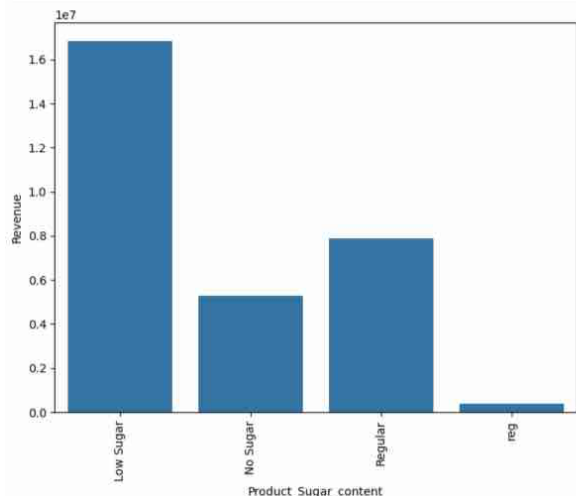
```
df_revenue1 = data.groupby(["Product_Type", as_index=False]  
    "Product_Store_Sales_Total"  
).sum()  
plt.figure(figsize=[14, 8])  
plt.xticks(rotation=90)  
a = sns.barplot(x=df_revenue1.Product_Type, y=df_revenue1.Product_Store_Sales_Total)  
a.set_xlabel("Product_Types")  
a.set_ylabel("Revenue")  
plt.show()
```



- **Top of the range.** Fruits and Vegetables lead at Rs. 4.30m, then Snack Foods Rs. 3.99m, Dairy and Frozen Foods Rs. 2.81m each, and Household Rs. 2.56m.
- **Bottom of the range.** Seafood Rs. 0.27m, Breakfast Rs. 0.36m and Starchy Foods Rs. 0.52m together contribute less than Snack Foods alone.
- **The key nuance.** Average revenue per line is nearly identical across all 16 categories (Rs. 3,416 to Rs. 3,679). The totals differ because of how many lines each category carries, not because any category sells better per item.
- **So what.** Category revenue is a shelf-count story. Growth comes from range width in proven categories, and the fresh and impulse leaders are where a stockout costs the most.

Bivariate: Revenue by Sugar Content

```
df_revenue2 = data.groupby(["Product_Sugar_Content"], as_index=False)[
    "Product_Store_Sales_Total"
].sum() #Complete the code to perform a groupby on Product_Sugar_Content and select Product_Store_Sales_Total
plt.figure(figsize=[8, 6])
plt.xticks(rotation=90)
b = sns.barplot(
    x=df_revenue2.Product_Sugar_Content, y=df_revenue2.Product_Store_Sales_Total
)
b.set_xlabel("Product_Sugar_Content")
b.set_ylabel("Revenue")
plt.show()
```

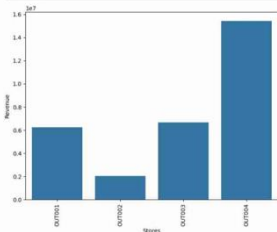


- **What the totals show.** Low Sugar Rs. 16.8m, Regular Rs. 7.9m, No Sugar Rs. 5.3m, and the stray "reg" label Rs. 0.38m.
- **What the averages show.** Revenue per line is flat: Low Sugar Rs. 3,444, No Sugar Rs. 3,469, Regular Rs. 3,502 - a spread of under Rs. 60.
- **Data issue, visible in the chart.** "reg" appears as a fourth bar. It is the same category as Regular and splits Rs. 0.38m of revenue away from it, understating Regular by roughly 5%.
- **So what.** Sugar content is not a demand driver, it is a mix descriptor: Low Sugar leads only because it is 55.7% of the range. Merge the label at source, and do not expect a health-positioning shift to move revenue on its own.

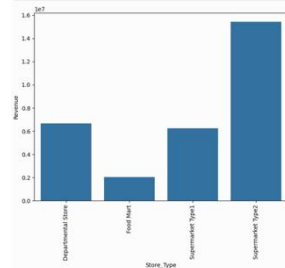
Bivariate: Revenue by Outlet and Format

Let us see from which type of stores and locations the revenue generation is more.

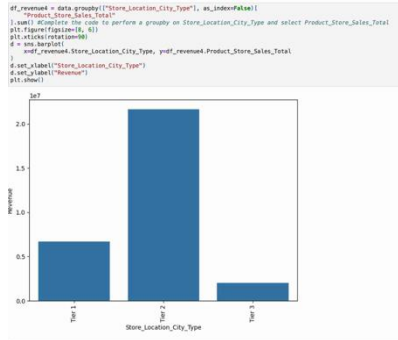
```
df_store_revenue = data.groupby(["Store_ID", "is_Index=False"])
df_store_revenue = df_store_revenue.groupby("Store_ID")
df_store_revenue = df_store_revenue["Revenue"]
df_store_revenue = df_store_revenue["Revenue"]
df_store_revenue = df_store_revenue["Revenue"]
df_store_revenue = df_store_revenue["Revenue"]
df_store_revenue = df_store_revenue["Revenue"]
df_store_revenue = df_store_revenue["Revenue"]
df_store_revenue = df_store_revenue["Revenue"]
df_store_revenue = df_store_revenue["Revenue"]
df_store_revenue = df_store_revenue["Revenue"]
```



```
df_revenue5 = data.groupby(["Store_Type", "is_Index=False"])
df_revenue5 = df_revenue5.groupby("Store_Type")
df_revenue5 = df_revenue5["Revenue"]
df_revenue5 = df_revenue5["Revenue"]
df_revenue5 = df_revenue5["Revenue"]
df_revenue5 = df_revenue5["Revenue"]
df_revenue5 = df_revenue5["Revenue"]
df_revenue5 = df_revenue5["Revenue"]
df_revenue5 = df_revenue5["Revenue"]
df_revenue5 = df_revenue5["Revenue"]
```



- **Total revenue by outlet.** OUT004 Rs. 15.43m, OUT003 Rs. 6.67m, OUT001 Rs. 6.22m, OUT002 Rs. 2.03m. OUT004 alone is half of all revenue.
- **Format tells the same story.** Supermarket Type2 Rs. 15.4m, Departmental Store Rs. 6.67m, Supermarket Type1 Rs. 6.22m, Food Mart Rs. 2.03m - because each format is one outlet.
- **Total versus per-product.** OUT004 leads on total but sits third on revenue per product (Rs. 3,299). OUT003 is the reverse: Rs. 6.67m total but the best per product at Rs. 4,947.
- **So what.** Judge outlets on both measures. OUT004 is the volume engine and the priority for availability; OUT003 is the productivity benchmark whose pricing and merchandising practices are worth copying.

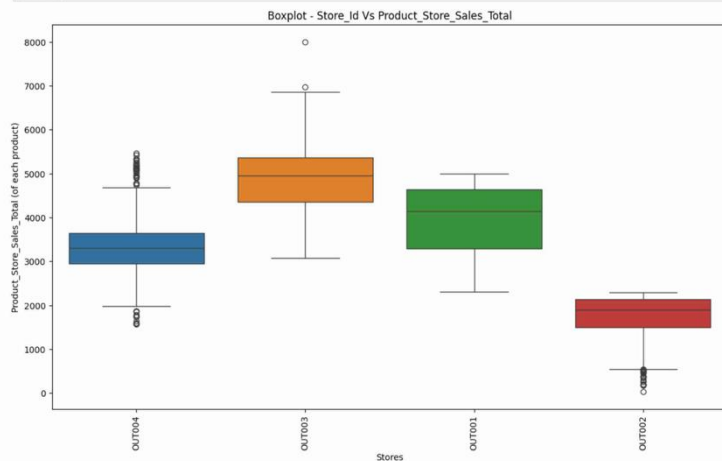


- Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Bivariate: Revenue Spread by Outlet

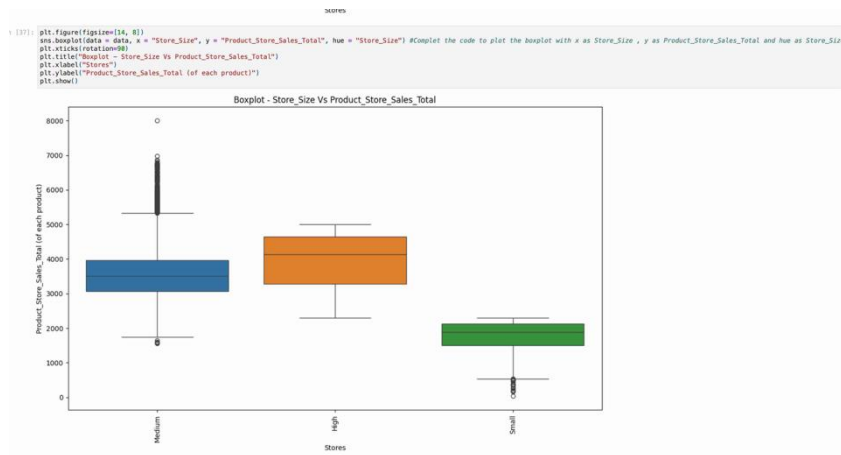
Let's check the distribution of our target variable i.e Product_Store_Sales_Total with the other categorical columns

```
[35]: plt.figure(figsize=(14, 8))
sns.boxplot(data=data, x="Store_Id", y="Product_Store_Sales_Total", hue = "Store_Id")
plt.xticks(rotation=90)
plt.title("Boxplot - Store_Id Vs Product_Store_Sales_Total")
plt.xlabel("Stores")
plt.ylabel("Product_Store_Sales_Total (of each product)")
plt.show()
```



- **Four distinct revenue profiles.** Median revenue per product runs OUT003 about Rs. 4,950, OUT001 about Rs. 4,150, OUT004 about Rs. 3,300 and OUT002 about Rs. 1,900.
- **Different consistency, too.** OUT003 and OUT001 have wide, high boxes with few outliers. OUT004 has a tight box but a long tail of high-value outliers, and OUT002 sits low and narrow.
- **Why the boxplot matters more than the totals.** The bars showed OUT004 winning on volume; the boxes show it is the least valuable per product of the three larger stores. Its revenue comes from breadth of range, not basket value.
- **So what.** Set forecast tolerances per outlet rather than one global band: OUT002 needs a narrow low-value band, while OUT004's outlier tail means safety stock on its premium lines.

Bivariate: Revenue Spread by Store Size

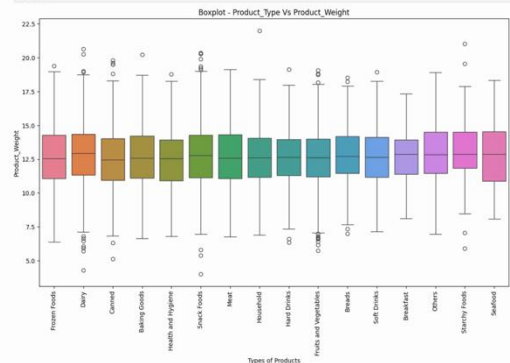


- **High beats Medium on the median.** High sits near Rs. 4,140 per product, Medium Rs. 3,480, Small Rs. 1,880 - the opposite of the totals ranking.
- **Medium carries the tail.** A dense band of outliers above Rs. 5,300, including the Rs. 8,000 maximum, from OUT004's wide range.
- **Small is constrained.** Tops out near Rs. 2,300 with a floor near zero.
- **So what.** Size predicts revenue per product opposite to total revenue: copy High-format practice, treat Medium's tail as a stocking risk.

Bivariate: Product Weight Profile

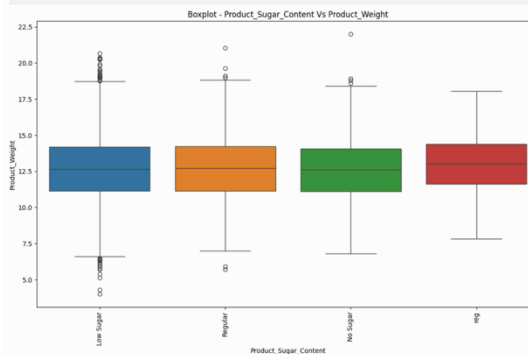
Let's now try to find out some relationship between the other columns

```
[10]: plt.figure(figsize=[14, 8])
sns.boxplot(data = data, x = "Product_Type", y = "Product_Weight", hue = "Product_Type") #Complete the code to plot the boxplot with x as Product_Type , y as Product_Weight and hue as Product_Type
plt.xticks(rotation=90)
plt.title("Boxplot - Product_Type Vs Product_Weight")
plt.xlabel("Types of Products")
plt.ylabel("Product_Weight")
plt.show()
```



Let's find out whether there is some relationship between the weight of the product and its sugar content

```
[11]: plt.figure(figsize=[14, 8])
sns.boxplot(data = data, x = "Product_Sugar_Content", y = "Product_Weight", hue = "Product_Sugar_Content") #Complete the code to plot the boxplot with x as Product_Sugar_Content , y as Product_Weight and hue as Product_Sugar_Content
plt.xticks(rotation=90)
plt.title("Boxplot - Product_Sugar_Content Vs Product_Weight")
plt.xlabel("Product_Sugar_Content")
plt.ylabel("Product_Weight")
plt.show()
```

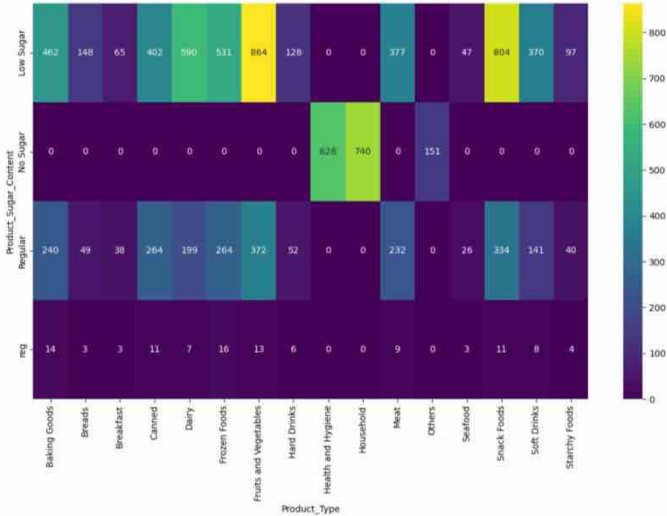


- **Constant across all 16 categories.** Medians sit between 12.5 and 13.0 with near-identical boxes.
- **Constant across sugar levels too.** Low, Regular and No Sugar all centre near 12.6.
- **So what.** Weight carries no category or nutrition signal, so it survives as an independent predictor; logistics assumptions can stay uniform across the range.

Sugar Content by Product Category

Let's analyze the sugar content of different product types

```
plt.figure(figsize=(14, 8))
sns.heatmap(
    pd.crosstab(data["Product_Sugar_Content"], data["Product_Type"]),
    annot=True,
    fmt="g",
    cmap="viridis",
)
plt.ylabel("Product_Sugar_Content")
plt.xlabel("Product_Type")
plt.show()
```

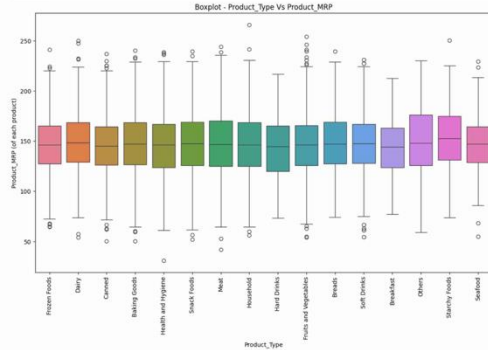


- **A structural pattern.** "No Sugar" appears in only three categories: Health and Hygiene 628, Household 740, Others 151.
- **Those three are the non-consumables.** 1,519 items - the NC prefix count - so it is a not-applicable flag.
- **Within food, Low Sugar leads.** Fruits and Vegetables 864 vs 372; Snack Foods 804 vs 334.
- **Issue.** "reg" is spread across 13 categories at 3 to 16 items each, so it never dominates a cell.
- **So what.** Treat sugar as two facts: food vs non-food, then low vs regular within food.

Bivariate: Price by Category and Outlet

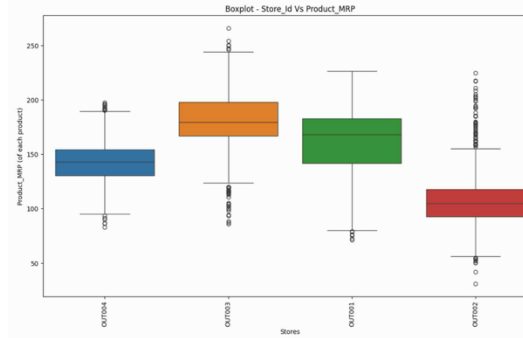
Different product types have different prices. Let's analyze the trend.

```
1 | plt.figure(figsize=(14, 8))
2 | sns.boxplot(data = data, x = "Product_Type", y = "Product_MRP", hue = "Product_Type") #Complete the code to plot a boxplot with x as Product_Type , y as Product_MRP and hue as Product_Type
3 | plt.xticks(rotation=90)
4 | plt.title("Boxplot - Product_Type Vs Product_MRP")
5 | plt.xlabel("Product_Type")
6 | plt.ylabel("Product_MRP (of each product)")
7 | plt.show()
```



Let's find out how the Product_MRP varies with the different stores

```
1 | plt.figure(figsize=(14, 8))
2 | sns.boxplot(data = data, x = "Store_Id", y = "Product_MRP", hue = "Store_Id") #Complete the code to plot the boxplot with x as Store_Id , y as Product_MRP and hue as Store_Id
3 | plt.xticks(rotation=90)
4 | plt.title("Boxplot - Store_Id Vs Product_MRP")
5 | plt.xlabel("Store_Id")
6 | plt.ylabel("Product_MRP (of each product)")
7 | plt.show()
```



- **Price does not vary by category (left).** All 16 medians sit between Rs. 145 and Rs. 153.
- **Price varies sharply by outlet (right).** Medians: OUT003 Rs. 180, OUT001 Rs. 168, OUT004 Rs. 143, OUT002 Rs. 105.
- **This explains the revenue gap.** Price mix, not category mix, separates the outlets.
- **So what.** Test trading OUT002 and OUT004 up a price band on proven lines.

Store Deep Dive: OUT001

Let's delve deeper and do a detailed analysis of each of the stores.

OUT001

```
43]: data.loc[data["Store_Id"] == "OUT001"].describe(include="all").T
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
Product_Id	1586	1586	FD5075	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product_Weight	1586.0	NaN	NaN	NaN	13.458865	2.064975	6.16	12.0525	13.96	14.95	17.97
Product_Sugar_Content	1586	4	Low Sugar	845	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product_Allocated_Area	1586.0	NaN	NaN	NaN	0.068768	0.047131	0.004	0.033	0.0565	0.094	0.295
Product_Type	1586	16	Snack Foods	202	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product_MRP	1586.0	NaN	NaN	NaN	160.514054	30.359059	71.35	141.72	168.32	182.9375	226.59
Store_Id	1586	1	OUT001	1586	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Store_Establishment_Year	1586.0	NaN	NaN	NaN	1987.0	0.0	1987.0	1987.0	1987.0	1987.0	1987.0
Store_Size	1586	1	High	1586	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Store_Location_City_Type	1586	1	Tier 2	1586	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Store_Type	1586	1	Supermarket Type1	1586	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product_Store_Sales_Total	1586.0	NaN	NaN	NaN	3923.778802	904.62901	2300.56	3285.51	4139.645	4639.4	4997.63

Observations

- OUT001 is a store of Supermarket Type 1 which is located in a Tier 2 city and has store size as high. It was established in 1987.
- OUT001 has sold products whose MRP range from 71 to 227.
- Snack Foods have been sold the highest number of times in OUT001.
- The revenue generated from each product at OUT001 ranges from 2300 to 5000.

```
data.loc[data["Store_Id"] == "OUT001", "Product_Store_Sales_Total"].sum()
```

6223113.18

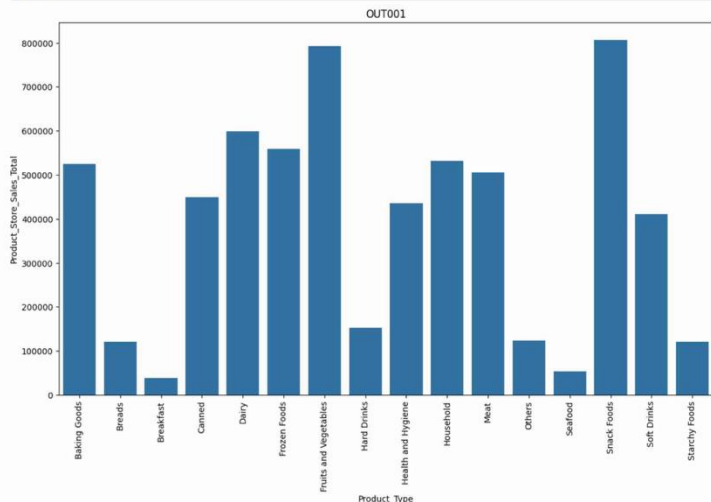
OUT001 has generated total revenue of 6223113 from the sales of goods.

- **Profile.** Supermarket Type 1, High size, Tier 2, opened 1987 - the oldest outlet. 1,586 records.
- **Assortment.** MRP averages Rs. 160.51 (Rs. 71 to 227); Snack Foods most stocked at 202 lines.
- **Revenue.** Rs. 6,223,113 total; Rs. 3,924 per product, spread Rs. 2,301 to Rs. 4,998.
- **Standout trait.** No product earns below Rs. 2,300 - the highest revenue floor of any outlet.
- **So what.** The reliability benchmark and the lowest-risk site for a replenishment pilot.

OUT001: Revenue by Category

OUT001 has generated total revenue of 6223113 from the sales of goods.

```
In [45]: df_OUT001 = {
data.loc[data["Store_Id"] == "OUT001"]
.groupby(["Product_Type", as_index=False])["Product_Store_Sales_Total"]
.sum()
}
plt.figure(figsize=(14, 8))
plt.xticks(rotation=90)
plt.xlabel("Product_Type")
plt.ylabel("Product_Store_Sales_Total")
plt.title("OUT001")
sns.barplot(x=df_OUT001.Product_Type, y=df_OUT001.Product_Store_Sales_Total)
plt.show()
```



- **Two categories lead.** Snack Foods about Rs. 808k and Fruits and Vegetables about Rs. 794k - together a quarter of the store's Rs. 6.22m.
- **A solid middle.** Dairy Rs. 600k, Frozen Foods Rs. 560k, Household Rs. 532k, Baking Goods Rs. 525k and Meat Rs. 506k all clear half a million.
- **A thin tail.** Breakfast Rs. 39k and Seafood Rs. 53k are negligible; Breads, Starchy Foods and Others sit near Rs. 120k each.
- **Consistent with the estate.** The same two categories lead company-wide, so OUT001 is not an outlier in mix - it simply sells the standard range at a higher price point.
- **So what.** Protect Snack Foods and Fresh availability here first; the bottom five categories are candidates for range rationalisation to free shelf and working capital.

Store Deep Dive: OUT002

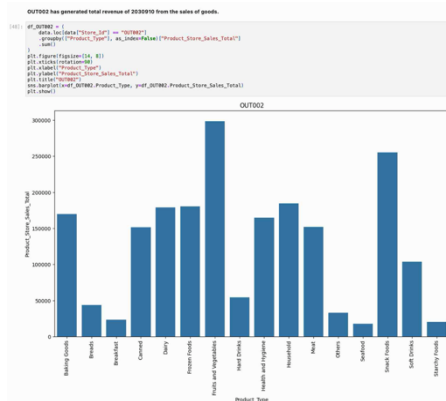
OUT002

```
[46]: data.loc[data["Store_ID"] == "OUT002"].describe(include="all").T
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
Product_ID	1152	1152	NCT192	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product_Weight	1152.0	NaN	NaN	NaN	9.91241	1.799846	4.0	8.7675	9.795	10.89	19.82
Product_Sugar_Content	1152	4	Low Sugar	639	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product_Allocated_Area	1152.0	NaN	NaN	NaN	0.067747	0.047587	0.006	0.031	0.0545	0.09525	0.292
Product_Type	1152	16	Fruits and Vegetables	168	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product_MRP	1152.0	NaN	NaN	NaN	107.080634	24.912353	31.0	82.8276	104.675	117.8175	234.93
Store_ID	1152	1	OUT002	1152	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Store_Establishment_Year	1152.0	NaN	NaN	NaN	1998.0	0.0	1998.0	1998.0	1998.0	1998.0	1998.0
Store_Size	1152	1	Small	1152	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Store_Location_City_Type	1152	1	Tier 3	1152	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Store_Location_Type	1152	1	Food Mart	1152	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product_Store_Sales_Total	1152.0	NaN	NaN	NaN	1762.942485	462.862431	33.0	1495.4725	1889.495	2133.6225	2299.63

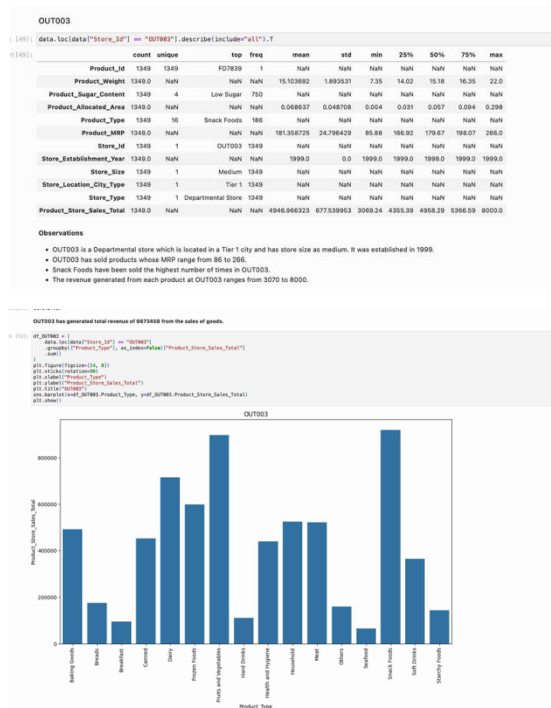
Observations

- OUT002 is a food mart which is located in a Tier 3 city and has store size as small. It was established in 1998.
- OUT002 has sold products whose MRP range from 31 to 235.
- Fruits and vegetables have been sold the highest number of times in OUT002.
- The revenue generated from each product at OUT002 ranges from 33 to 2300.



- **Profile.** Food Mart, Small size, Tier 3 city, opened 1998. 1,152 records - the smallest outlet.
- **A different range entirely.** MRP averages Rs. 107.08 and weight 9.91, both the lowest of the four outlets. It stocks smaller, cheaper products.
- **Revenue.** Rs. 2,030,910 total; Rs. 1,763 per product, from Rs. 33 up to a ceiling of Rs. 2,300.
- **Category mix.** Fruits and Vegetables Rs. 299k and Snack Foods Rs. 255k lead; Seafood Rs. 18k and Starchy Foods Rs. 21k are negligible.
- **Issue.** The Rs. 33 floor is 70x below the Rs. 2,300 ceiling, so this store contains the widest relative revenue range - the hardest to forecast in percentage terms.
- **So what.** Do not judge OUT002 on absolute revenue. It serves a lower-income Tier 3 catchment and needs its own localized assortment, price ladder and forecast tolerance.

Store Deep Dive: OUT003



- **Profile.** Departmental Store, Medium size, Tier 1 city, opened 1999. 1,349 records.
- **The premium range.** MRP averages Rs. 181.36 and weight 15.10 - both the highest of the four outlets. It stocks the largest, most expensive products.
- **Revenue.** Rs. 6,673,458 total; Rs. 4,947 per product, from Rs. 3,069 to the Rs. 8,000 maximum in the whole dataset.
- **Category mix.** Snack Foods Rs. 920k and Fruits and Vegetables Rs. 890k lead, then Dairy Rs. 715k. Seafood Rs. 65k trails.
- **The standout trait.** Its revenue floor of Rs. 3,069 is higher than the average product at every other outlet. Even its worst-selling line beats OUT002's best.
- **So what.** OUT003 is the productivity template: premium price band, larger pack sizes, Tier 1 catchment. Copy its price ladder and pack-size strategy before copying its category mix.

[illegible]

- Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Four Outlets, Four Distinct Segments

Outlet	Format	Tier	Records	Avg MRP	Avg weight	Rev / product	Total revenue
OUT002	Food Mart	3	1,152	107.08	9.91	1,763	Rs. 2.03m
OUT004	Supermarket 2	2	4,676	142.40	12.35	3,299	Rs. 15.43m
OUT001	Supermarket 1	2	1,586	160.51	13.46	3,924	Rs. 6.22m
OUT003	Departmental	1	1,349	181.36	15.10	4,947	Rs. 6.67m

- **The pattern the deep dives reveal.** Price, pack weight and revenue per product all move together in the same order across the estate: OUT002 (Rs. 107 / 9.9 / Rs. 1,763), OUT004 (Rs. 142 / 12.3 / Rs. 3,299), OUT001 (Rs. 161 / 13.5 / Rs. 3,924), OUT003 (Rs. 181 / 15.1 / Rs. 4,947).
- **Why this matters for the model.** The "weight drives revenue" correlation of 0.74 is largely a store-segment effect: heavier products sit in higher-priced stores. Weight is acting as a proxy for which outlet a product belongs to.
- **Recommendation.** Plan by segment, not by one national rule: a value range for Tier 3, a mid-market scale range for Tier 2, and a premium range for Tier 1. Pilot forecasting at OUT001 (most predictable) and OUT004 (largest), then extend.

Revenue by Category and Store

OUT004 has generated the highest revenue from the sale of fruits and vegetables (~2500000) followed by snack foods (~2)
Let's find out the revenue generated by the stores from each of the product types.

```

In [101]: df1 = data.groupby(["Product_Type", "Store_ID"], as_index=False)[
        "Product_Sales_Total"
        ].sum()
df1

```

Store_ID	Product_Type	Store_ID	Product_Sales_Total
0	Baking Goods	OUT001	525131.04
1	Baking Goods	OUT002	169860.50
2	Baking Goods	OUT003	49109.30
3	Baking Goods	OUT004	1585586.36
4	Breads	OUT001	121276.09
5	Breads	OUT002	43419.47
6	Breads	OUT003	175391.93
7	Breads	OUT004	374856.75
8	Breakfast	OUT001	38161.10
9	Breakfast	OUT002	23396.10
10	Breakfast	OUT003	96534.08
11	Breakfast	OUT004	204939.13
12	Canned	OUT001	440016.38
13	Canned	OUT002	1514827.66
14	Canned	OUT003	452445.37
15	Canned	OUT004	1247153.50
16	Dairy	OUT001	595767.62
17	Dairy	OUT002	178888.18
18	Dairy	OUT003	715814.94
19	Dairy	OUT004	1318447.30
20	Frozen Foods	OUT001	558356.81
21	Frozen Foods	OUT002	160395.95
22	Frozen Foods	OUT003	907608.42
23	Frozen Foods	OUT004	1473519.65
24	Fruits and Vegetables	OUT001	792982.59
25	Fruits and Vegetables	OUT002	288503.56
26	Fruits and Vegetables	OUT003	897437.46
27	Fruits and Vegetables	OUT004	2311899.66
28	Hard Drinks	OUT001	152920.74
29	Hard Drinks	OUT002	545381.85
30	Hard Drinks	OUT003	110760.80

31	Hard Drinks	OUT004	3075851.73
32	Health and Hygiene	OUT001	435005.31
33	Health and Hygiene	OUT002	164660.81
34	Health and Hygiene	OUT003	439139.18
35	Health and Hygiene	OUT004	1124901.91
36	Household	OUT001	531371.38
37	Household	OUT002	184665.65
38	Household	OUT003	523981.64
39	Household	OUT004	1324721.50
40	Meat	OUT001	505667.28
41	Meat	OUT002	151800.01
42	Meat	OUT003	520939.68
43	Meat	OUT004	950604.97
44	Others	OUT001	123977.59
45	Others	OUT002	32835.73
46	Others	OUT003	199963.75
47	Others	OUT004	224719.75
48	Seafood	OUT001	52536.84
49	Seafood	OUT002	17663.35
50	Seafood	OUT003	65337.48
51	Seafood	OUT004	136866.37
52	Snack Foods	OUT001	806142.34
53	Snack Foods	OUT002	255317.87
54	Snack Foods	OUT003	91810.44
55	Snack Foods	OUT004	2009026.70
56	Soft Drinks	OUT001	410548.69
57	Soft Drinks	OUT002	103808.35
58	Soft Drinks	OUT003	365046.30
59	Soft Drinks	OUT004	917641.38
60	Starchy Foods	OUT001	120443.98
61	Starchy Foods	OUT002	20044.98
62	Starchy Foods	OUT003	143538.60
63	Starchy Foods	OUT004	234746.89

- **A clean, consistent ranking.** Across all 16 categories the order never changes: OUT004 highest, then OUT003 or OUT001, then OUT002 lowest. No category bucks the store hierarchy.
- **The gap is wider than volume alone.** In Fruits and Vegetables OUT004 earns Rs. 2.31m against OUT002's Rs. 0.30m - 7.7x, while OUT004 has only 4.1x the records. Higher price per line accounts for the rest.
- **A merchandising nuance.** OUT003 beats OUT001 on fresh and chilled (Snack Foods Rs. 919k vs 806k, Dairy Rs. 716k vs 599k) but loses on ambient and drinks (Baking Rs. 492k vs 525k, Soft Drinks Rs. 365k vs 411k).
- **So what.** Because the ranking is stable, category-level forecasts can be modelled once and scaled by a store factor. The OUT001 versus OUT003 split is the exception worth planning by hand: they have genuinely different category strengths.

Revenue by Sugar Content and Store

Sugar level	OUT001	OUT002	OUT003	OUT004	Total
Low Sugar	3,300,835	1,156,759	3,706,903	8,658,909	16,823,406
Regular	1,749,445	472,113	1,743,566	3,902,548	7,867,672
No Sugar	1,090,354	382,162	1,123,085	2,674,343	5,269,944
"reg" (mislabelled)	82,480	19,876	99,903	191,784	394,043

- **Same store hierarchy again.** Within every sugar level: OUT004, then OUT003 or OUT001, then OUT002.
- **One reversal.** On Low Sugar, OUT003 earns Rs. 3.71m against OUT001's Rs. 3.30m despite 237 fewer lines - premium pricing at work.
- **The data issue, quantified.** The stray "reg" label carries Rs. 394,043, all reported outside Regular.
- **So what.** Sugar adds no forecasting signal beyond store and price; merge "reg" into Regular at source.

EDA Insights and Recommendations

WHAT THE DATA TELLS US

- Price is the strongest revenue driver (MRP 0.79); display area has none (-0.001).
- Store segment sets the level: price, pack weight and revenue per product rank identically across the four outlets.
- Price mix, not category mix, separates outlets - all 16 category medians sit at Rs. 145 to 153.
- The same store ranking repeats in every category and every sugar level.
- Totals and per-product revenue rank inversely: footprint drives one, productivity the other.

WHAT WE RECOMMEND

- Lead assortment planning with price band, not shelf space.
- Plan by segment: value range for Tier 3, mid-market scale for Tier 2, premium for Tier 1.
- Copy OUT003's price ladder and OUT001's ambient and drinks range.
- Protect Fruits and Vegetables and Snack Foods availability first.
- Reassign display area by category sales velocity and prove it with a store test.
- Pilot forecasting at OUT001 (most predictable) and OUT004 (largest).

Data Issues and Modelling Implications

DATA ISSUES TO FIX AT SOURCE

- Merge "reg" into Regular: Rs. 394,043 of revenue is reported outside Regular today.
- Split sugar content into food versus non-food - "No Sugar" tags all 1,519 non-consumables.
- Drop Product_Id as a feature (unique on all 8,763 rows); keep only its FD, DR, NC prefix.
- Treat establishment year as a store label, not a number - it holds only four values.

WHAT IT MEANS FOR THE MODEL

- Weight's 0.74 correlation is partly a store proxy, so the feature set will not transfer to a new outlet.
- Store size, tier, type and year are perfectly collinear: the model learns four store profiles, not general effects.
- OUT004 supplies 53% of rows, so overall accuracy reflects its mid-market pattern. Track error per outlet.
- OUT002 spans Rs. 33 to Rs. 2,300 - expect its percentage error to be the worst of the four.
- The target is symmetric and the data complete, so no transformation or imputation is required.

Preprocessing: Feature Engineering

Data Preprocessing

Replacing the values in the Product_Sugar_Content column

We can observe that in the Product_Sugar_Content column, there are 3 types - Low Sugar, Regular and reg.

It seems quite obvious that Regular and reg are referring to the same category. So let's replace reg with Regular.

```
# Replacing reg with Regular
data.Product_Sugar_Content.replace(to_replace=["reg"], value=["Regular"], inplace=True)
```

```
data.Product_Sugar_Content.value_counts()
```

```
Low Sugar    4885
Regular      2359
No Sugar     1519
Name: Product_Sugar_Content, dtype: int64
```

Grouping Product Types into Perishables and Non-Perishables.

We have 16 different product types in our dataset.

So let us make two broad categories, perishables and non-perishables, in order to reduce the number of product types

```
[100]: perishables = [
        "Dairy",
        "Meat",
        "Fruits and Vegetables",
        "Breakfast",
        "Breads",
        "Seafood",
    ]

[101]: def change(x):
        if x in perishables:
            return "Perishables"
        else:
            return "Non Perishables"

[102]: data['Product_Type_Category'] = data['Product_Type'].apply(change)

[103]: data.head()
```

	Product_Id	Product_Weight	Product_Sugar_Content	Product_Allocated_Area	Product_Type	Product_MRP	Store_Id	Store_Establishment_Year	Store_Size	Store_Location_City_Type	Store_Type	Product_Store_Sales_Total	Price
0	FD0114	12.66	Low Sugar	0.027	Frozen Foods	117.08	OUT004	2009	Medium	Tier 2	Supermarket Type2	2842.40	
1	FD0789	16.64	Low Sugar	0.144	Dairy	174.43	OUT003	1999	Medium	Tier 1	Departmental Store	4830.02	
2	FD0675	14.28	Regular	0.031	Canned	162.08	OUT001	1987	High	Tier 2	Supermarket Type1	4130.16	
3	FD0623	12.10	Low Sugar	0.112	Baking Goods	186.31	OUT001	1987	High	Tier 2	Supermarket Type1	4132.18	
4	NC1180	9.57	No Sugar	0.010	Health and Hygiene	123.87	OUT002	1998	Small	Tier 3	Food Mart	2279.36	

- **Label fix first.** "reg" became "Regular": Low Sugar 4,885, Regular 2,359, No Sugar 1,519 - recovering Rs. 394,043.
- **Product_Id_char.** FD covers the 11 food types, NC the three non-consumables, DR the drinks. The full ID was dropped.
- **Store_Age_Years.** 2025 minus opening year gives 16, 26, 27 and 38 years - more interpretable than a calendar year.
- **Product_Type_Category.** 16 types grouped into Perishables and Non Perishables, cutting encoded columns while keeping spoilage risk.
- **So what.** EDA showed category and sugar detail add little to revenue, so the engineering keeps only what the business acts on.

Preprocessing: Outlier Check

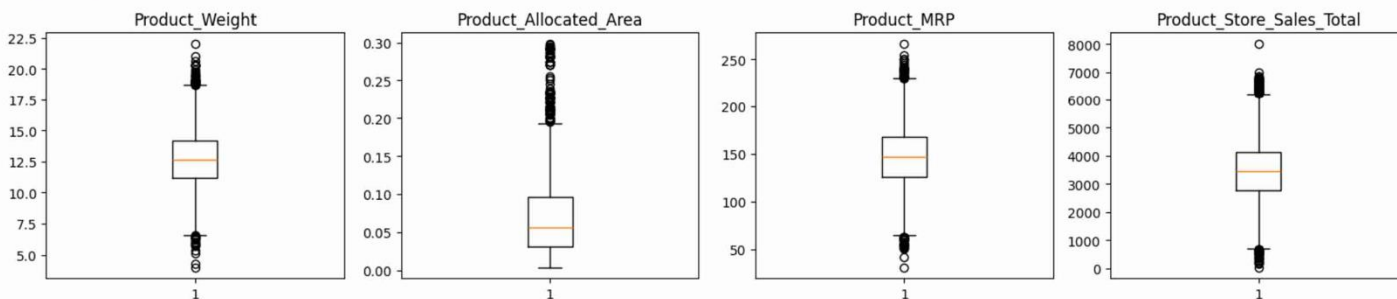
Outlier Check

```
701: # outlier detection using boxplot
numeric_columns = data.select_dtypes(include=np.number).columns.tolist()
numeric_columns.remove("Store_Establishment_Year")
numeric_columns.remove("Store_Age_Years")

plt.figure(figsize=(15, 12))

for i, variable in enumerate(numeric_columns):
    plt.subplot(4, 4, i + 1)
    plt.boxplot(data[variable], whis=1.5)
    plt.tight_layout()
    plt.title(variable)

plt.show()
```



- Tested at $1.5 \times \text{IQR}$ on all four numeric fields. Tails appear on every one; display area is the most skewed.
- **Decision: no treatment applied.** Rationale on the next slide.

Why Outliers Were Kept, Not Treated

THREE REASONS FOR NO TREATMENT

- They are real trade. The Rs. 8,000 maximum is a genuine premium line at OUT003; the Rs. 33 minimum a genuine low-value sale at OUT002.
- They are store segments, not errors. The revenue and MRP tails line up with outlet price bands, so they carry signal.
- Tree models split on rank order, so monotone extremes do not distort fitting the way they would in a linear model.

BUSINESS IMPACT

- Capping would have flattened exactly the high-value lines the business most wants forecast accurately.
- It would also have understated OUT003's revenue potential, the outlet being held up as the productivity benchmark.
- Display area is the one genuinely skewed field, but EDA showed it has no link to revenue, so its tail is harmless.
- Trade-off accepted: keeping the tails widens the error band on extreme lines, which is why safety stock and human review stay in the loop.

Preprocessing: Data Preparation

```

Let's remove the columns that are not required.
740: data = data.drop(["Product_Id",
741:                  "Product_Type",
742:                  "Store_Id",
743:                  "Store_Establishment_Year"], axis=1) #Complete the code to drop the columns "Product_Id","Product_Type","Store_Id","Store_Establishment_Year"

751: data.shape
752: (8763, 11)

761: data.head()
762:
763:
764:
765:
766:
767:
768:
769:
770:
771:
772:
773:
774:
775:
776:
777:
778:
779:
780:
781:
782:
783:
784:
785:
786:
787:
788:
789:
790:
791:
792:
793:
794:
795:
796:
797:
798:
799:
800:
801:
802:
803:
804:
805:
806:
807:
808:
809:
810:
811:
812:
813:
814:
815:
816:
817:
818:
819:
820:
821:
822:
823:
824:
825:
826:
827:
828:
829:
830:
831:
832:
833:
834:
835:
836:
837:
838:
839:
840:
841:
842:
843:
844:
845:
846:
847:
848:
849:
850:
851:
852:
853:
854:
855:
856:
857:
858:
859:
860:
861:
862:
863:
864:
865:
866:
867:
868:
869:
870:
871:
872:
873:
874:
875:
876:
877:
878:
879:
880:
881:
882:
883:
884:
885:
886:
887:
888:
889:
890:
891:
892:
893:
894:
895:
896:
897:
898:
899:
900:
901:
902:
903:
904:
905:
906:
907:
908:
909:
910:
911:
912:
913:
914:
915:
916:
917:
918:
919:
920:
921:
922:
923:
924:
925:
926:
927:
928:
929:
930:
931:
932:
933:
934:
935:
936:
937:
938:
939:
940:
941:
942:
943:
944:
945:
946:
947:
948:
949:
950:
951:
952:
953:
954:
955:
956:
957:
958:
959:
960:
961:
962:
963:
964:
965:
966:
967:
968:
969:
970:
971:
972:
973:
974:
975:
976:
977:
978:
979:
980:
981:
982:
983:
984:
985:
986:
987:
988:
989:
990:
991:
992:
993:
994:
995:
996:
997:
998:
999:

```

	Product_Weight	Product_Sugar_Content	Product_Allocated_Area	Product_MRP	Store_Size	Store_Location_City_Type	Store_Type	Product_Store_Sales_Total	Product_Id_Char	Store_Age_Years	Product_Type_Category
0	12.66	Low Sugar	0.027	117.08	Medium	Tier 2	Supermarket Type2	2642.40	FD	16	Non Perishables
1	16.54	Low Sugar	0.144	171.43	Medium	Tier 1	Departmental Store	4450.02	FD	26	Perishables
2	14.28	Regular	0.031	162.06	High	Tier 2	Supermarket Type1	4102.16	FD	38	Non Perishables
3	15.10	Low Sugar	0.112	186.31	High	Tier 2	Supermarket Type1	4102.16	FD	38	Non Perishables
4	9.57	No Sugar	0.010	123.67	Small	Tier 3	Food Mart	2279.36	NC	27	Non Perishables

```

771: # Separating features and the target column
772: X = data.drop("Product_Store_Sales_Total", axis=1) #Complete the code to drop the target variable
773: y = data["Product_Store_Sales_Total"] #Complete the code to select the target variable

774:
775:
776:
777:
778:
779:
780:
781:
782:
783:
784:
785:
786:
787:
788:
789:
790:
791:
792:
793:
794:
795:
796:
797:
798:
799:
800:
801:
802:
803:
804:
805:
806:
807:
808:
809:
810:
811:
812:
813:
814:
815:
816:
817:
818:
819:
820:
821:
822:
823:
824:
825:
826:
827:
828:
829:
830:
831:
832:
833:
834:
835:
836:
837:
838:
839:
840:
841:
842:
843:
844:
845:
846:
847:
848:
849:
850:
851:
852:
853:
854:
855:
856:
857:
858:
859:
860:
861:
862:
863:
864:
865:
866:
867:
868:
869:
870:
871:
872:
873:
874:
875:
876:
877:
878:
879:
880:
881:
882:
883:
884:
885:
886:
887:
888:
889:
890:
891:
892:
893:
894:
895:
896:
897:
898:
899:
900:
901:
902:
903:
904:
905:
906:
907:
908:
909:
910:
911:
912:
913:
914:
915:
916:
917:
918:
919:
920:
921:
922:
923:
924:
925:
926:
927:
928:
929:
930:
931:
932:
933:
934:
935:
936:
937:
938:
939:
940:
941:
942:
943:
944:
945:
946:
947:
948:
949:
950:
951:
952:
953:
954:
955:
956:
957:
958:
959:
960:
961:
962:
963:
964:
965:
966:
967:
968:
969:
970:
971:
972:
973:
974:
975:
976:
977:
978:
979:
980:
981:
982:
983:
984:
985:
986:
987:
988:
989:
990:
991:
992:
993:
994:
995:
996:
997:
998:
999:

```

- **Four columns dropped.** Product_Id, Product_Type, Store_Id and Store_Establishment_Year - 12 columns become 11.
- **Features and target split.** Sales becomes y; 10 columns become X - four numeric, six categorical.
- **Split 70:30, seed 1.** 6,134 training rows and 2,629 test rows, shuffled and reproducible.
- **Why hold out 30%.** With only four outlets the risk is memorising store profiles; a large test set is the check on that.
- **Issue.** The split is not stratified by store, so OUT004 dominates the test set as it does training.

Preprocessing: The Encoding Pipeline

Data Pre-processing Pipeline

```
In [80]: categorical_features = data.select_dtypes(include=['object', 'category']).columns.tolist()
categorical_features
```

```
Out[80]: ['Product_Sugar_Content',
'Store_Size',
'Store_Location_City_Type',
'Store_Type',
'Product_Id_char',
'Product_Type_Category']
```

```
In [81]: # Create a preprocessing pipeline for the categorical features

preprocessor = make_column_transformer(
    (Pipeline([['encoder', OneHotEncoder(handle_unknown='ignore')]]), categorical_features)
)
```

name: product_sugar_content, dtype: object

Exploring Patterns in Product_IDs

We can see that the Product_Id column has two characters followed by a number.

Let's delve deeper and see whether they are having any relationship with the other columns or not.

```
[90]: # Extracting the first two characters from the Product_Id column and storing it in another column
data['Product_Id_char'] = data['Product_Id'].str[:2]
data.head()
```

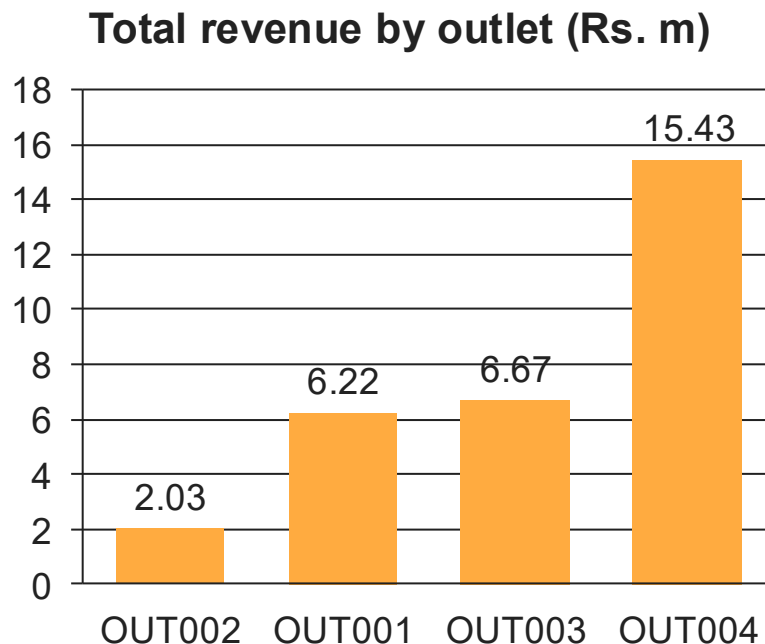
```
[90]:
```

	Product_Id	Product_Weight	Product_Sugar_Content	Product_Allocated_Area	Product_Type	Product_MRP	Store_Id	Store_Establishment_Year	Store_Size	Store_Location_City_Type	Store_Type	Product_Stores_Sales_Total	P
0	FD0114	12.66	Low Sugar	0.027	Frozen Foods	117.08	OUT004	2009	Medium	Tier 2	Supermarket Type2	2842.40	
1	FD7839	16.54	Low Sugar	0.144	Dairy	171.43	OUT003	1999	Medium	Tier 1	Departmental Store	4830.02	
2	FD0075	14.28	Regular	0.031	Canned	182.08	OUT001	1987	High	Tier 2	Supermarket Type1	4130.16	
3	FD0233	12.10	Low Sugar	0.112	Baking Goods	186.31	OUT001	1987	High	Tier 2	Supermarket Type1	4132.18	
4	NC1180	9.87	No Sugar	0.010	Health and Hygiene	123.67	OUT002	1998	Small	Tier 3	Food Mart	2279.30	

- **Six categorical features one-hot encoded.** The four numeric fields pass through untouched.
- **Bundled with the estimator.** One ColumnTransformer pipeline, so training and prediction share the same transformation.
- **handle_unknown = ignore.** An unseen category encodes as zeros instead of raising an error, so the API degrades gracefully.
- **One-hot, not ordinal.** Store_Size looks ordered but its revenue ranking is not, so an order would be false.
- **So what.** What was validated in the notebook is what runs behind the Flask endpoint.

Data Background and Contents

- **Where it comes from.** A single historical sales extract, SuperKart.csv, covering four operating outlets across Tier 1, 2 and 3 cities.
- **What one row is.** One product sold in one store, with the revenue it generated. 8,763 rows, each a distinct product line.
- **What it contains.** 12 original attributes: six describe the product, five describe the store, one is the target. Four are numeric measures and seven are text descriptors.
- **How much business it represents.** Rs. 30.36 million of revenue in total, ranging from Rs. 33 to Rs. 8,000 per product-store line.
- **So what.** This is the exact grain at which replenishment decisions are made, which is why the data supports forecasting directly - but it is a snapshot, not a time series.



What This Data Can and Cannot Tell Us

WHAT IT CAN ANSWER

- Expected revenue for a given product in a given store.
- Which outlets, categories and price bands generate the most revenue.
- How strongly price and pack size relate to revenue.
- Which product attributes do not matter - shelf space, sugar level, category detail.

WHAT IT CANNOT ANSWER (AND THE FIX)

- No dates, so no seasonality, trend or quarter-on-quarter view. Capture transaction dates.
- No cost or margin, so revenue only - not profit. Add cost price per line.
- No promotions, footfall or stock-out flags, so cause cannot be attributed. Log promo and out-of-stock events.
- Only four outlets, all attributes fixed per store, so forecasts will not transfer to a new site. Add outlets as they open.
- No customer or basket data, so no loyalty or cross-sell view.

Model Building: Metric and Approach

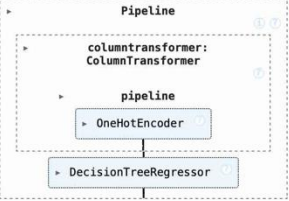
- **Primary metric: R-squared.** The business question is how much of the variation in product-store revenue we can explain, so a share-of-variance metric is the right headline. Adjusted R-squared is reported alongside it to penalise the 10 predictors.
- **Reported in rupees too.** RMSE and MAE convert error into money per product-store line, which is what an inventory planner actually needs. MAPE gives the same error as a percentage for comparing across outlets of different size.
- **Why not accuracy-style metrics.** This is a regression problem, not classification. There is no threshold to be right or wrong against - only distance from the true revenue.
- **Two models chosen.** Random Forest (bagging) and Gradient Boosting (boosting) - one of each ensemble family, so the comparison tests two genuinely different learning strategies rather than two variants of the same one.
- **Both built inside the pipeline.** Each estimator is wrapped with the one-hot encoder using `make_pipeline`, so encoding is fitted on training folds only and cannot leak into validation.

Model 1: Decision Tree Regressor

Decision Tree Model

```
In [83]: # Uncomment the below snippet of code if decision tree regressor is to be used
dtree = DecisionTreeRegressor(random_state=1)
dtree = make_pipeline(preprocessor,dtree)
dtree.fit(X_train, y_train)
```

Out[83]:



```
graph TD
    Pipeline[Pipeline] --> ColumnTransformer[columntransformer: ColumnTransformer]
    Pipeline --> DecisionTreeRegressor[DecisionTreeRegressor]
    ColumnTransformer --> pipeline[pipeline]
    pipeline --> OneHotEncoder[OneHotEncoder]
```

Checking model performance on training set

```
In [84]: # Uncomment the below snippet of code if decision tree regressor is to be used
dtree_model_train_perf = model_performance_regression(dtree, X_train, y_train)
dtree_model_train_perf
```

Out[84]:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	596.978222	468.965498	0.685033	0.684519	0.16569

Checking model performance on test set

```
In [85]: # Uncomment the below snippet of code if decision tree regressor is to be used
dtree_model_test_perf = model_performance_regression(dtree, X_test, y_test)
dtree_model_test_perf
```

Out[85]:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	615.933034	485.429583	0.668482	0.667215	0.187421

- **How it works.** A single tree splits the data on the feature that most reduces error at each node. It is the simplest of the five and serves as the baseline the ensembles must beat.
- **Training set.** RMSE Rs. 596.98, MAE Rs. 468.97, R-squared 0.6850, adjusted 0.6845, MAPE 16.57%.
- **Test set.** RMSE Rs. 615.93, MAE Rs. 485.43, R-squared 0.6685, adjusted 0.6672, MAPE 18.74%.
- **Read.** An unrestricted tree would normally overfit badly, yet the gap here is only 1.66 points of R-squared. That tells us the tree is finding broad, stable splits - almost certainly on MRP and store - rather than memorising rows.
- **Why it matters.** This is the result that makes the whole benchmark interesting: if one tree performs like the ensembles, the problem is not model complexity.

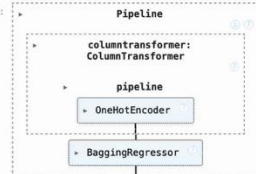
Model 2: Bagging Regressor

Bagging Regressor

In [86]: # Uncomment the below snippet of code if baggingregressor is to be used

```
bagging_regressor = BaggingRegressor(random_state=1)
bagging_regressor = make_pipeline(preprocessor, bagging_regressor)
bagging_regressor.fit(X_train, y_train)
```

Out [86]:



Checking model performance on training set

In [87]: # Uncomment the below snippet of code if baggingregressor is to be used

```
bagging_regressor_model_train_perf = model_performance_regression(bagging_regressor, X_train, y_train)
bagging_regressor_model_train_perf
```

Out [87]:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	597.064588	469.426208	0.684942	0.684428	0.165799

Checking model performance on test set

In [88]: # Uncomment the below snippet of code if baggingregressor is to be used

```
bagging_regressor_model_test_perf = model_performance_regression(bagging_regressor, X_test, y_test)
bagging_regressor_model_test_perf
```

Out [88]:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	615.866125	485.588892	0.668554	0.667288	0.18735

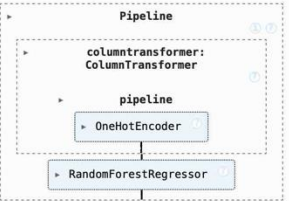
- **How it works.** Trains many trees on bootstrap samples of the training rows and averages them. Averaging reduces variance without changing the underlying tree logic.
- **Training set.** RMSE Rs. 597.06, MAE Rs. 469.43, R-squared 0.6849, adjusted 0.6844, MAPE 16.58%.
- **Test set.** RMSE Rs. 615.87, MAE Rs. 485.59, R-squared 0.66855, adjusted 0.6673, MAPE 18.74%.
- **Read.** Marginally the best test score of all five models and the smallest train-to-test gap at 0.0164 - but the margin over Gradient Boosting is 0.00003 of R-squared, which is noise, not a real advantage.
- **Why it matters.** Bagging's job is to cut variance. It barely moved the score, confirming variance was never the problem here.

Model 3: Random Forest Regressor

Random Forest Model

```
In [89]: # Uncomment the below snippet of code if random forest regressor is to be used
rf_estimator = RandomForestRegressor(random_state=1)
rf_estimator = make_pipeline(preprocessor, rf_estimator)
rf_estimator.fit(X_train, y_train)
```

Out[89]:



Checking model performance on training set

```
In [90]: # Uncomment the below snippet of code if random forest regressor is to be used
rf_estimator_model_train_perf = model_performance_regression(rf_estimator, X_train, y_train)
rf_estimator_model_train_perf
```

Out[90]:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	596.994959	468.87585	0.685016	0.684501	0.165674

Checking model performance on test set

```
In [91]: # Uncomment the below snippet of code if random forest regressor is to be used
rf_estimator_model_test_perf = model_performance_regression(rf_estimator, X_test, y_test)
rf_estimator_model_test_perf
```

Out[91]:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	615.906846	485.311027	0.66851	0.667244	0.187394

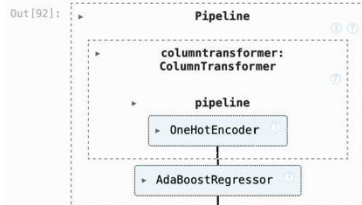
- **How it works.** Bagging plus a second source of randomness - each split considers only a random subset of features, which decorrelates the trees further.
- **Training set.** RMSE Rs. 596.99, MAE Rs. 468.88, R-squared 0.6850, adjusted 0.6845, MAPE 16.57%.
- **Test set.** RMSE Rs. 615.91, MAE Rs. 485.31, R-squared 0.66851, adjusted 0.6672, MAPE 18.74%.
- **Read.** Best MAE on both sets, so its typical error is the lowest even though R-squared ties with the others. Gap of 0.0165 - stable.
- **Why it matters.** This is the model taken forward, tuned and ultimately deployed. Feature subsampling also makes it the least sensitive to any single predictor, which matters when one field (MRP) dominates.

Model 4: AdaBoost Regressor

AdaBoost Regressor

In [92]: # Uncomment the below snippet of code if adaboost regressor is to be used

```
ab_regressor = AdaBoostRegressor(random_state=1)
ab_regressor = make_pipeline(preprocessor, ab_regressor)
ab_regressor.fit(X_train, y_train)
```



Checking model performance on training set

In [93]: # Uncomment the below snippet of code if adaboost regressor is to be used

```
ab_regressor_model_train_perf = model_performance_regression(ab_regressor, X_train, y_train)
ab_regressor_model_train_perf
```

Out[93]:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	627.778405	512.191947	0.651694	0.651126	0.172981

Checking model performance on test set

In [94]: # Uncomment the below snippet of code if adaboost regressor is to be used

```
ab_regressor_model_test_perf = model_performance_regression(ab_regressor, X_test, y_test)
ab_regressor_model_test_perf
```

Out[94]:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	647.483018	530.557049	0.633649	0.63225	0.193723

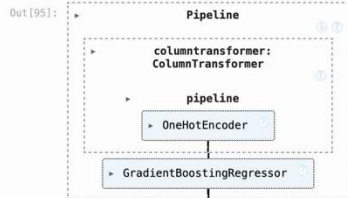
- **How it works.** Fits very shallow trees in sequence, each one weighted to focus on the rows the previous ones got wrong.
- **Training set.** RMSE Rs. 627.78, MAE Rs. 512.19, R-squared 0.6517, adjusted 0.6511, MAPE 17.30%.
- **Test set.** RMSE Rs. 647.48, MAE Rs. 530.56, R-squared 0.6336, adjusted 0.6323, MAPE 19.37%.
- **Read.** The clear laggard, and the only model weaker on training data too. That signals underfitting, not overfitting - its stumps are too shallow to capture a smooth price-to-revenue relationship.
- **Decision.** Eliminated before tuning. Rs. 647 RMSE is Rs. 32 worse per line than the alternatives, which on 8,763 lines is a material planning difference.

Model 5: Gradient Boosting Regressor

Gradient Boosting Regressor

In [95]: # Uncomment the below snippet of code if gradientboost regressor is to be used

```
gb_estimator = GradientBoostingRegressor(random_state=1)
gb_estimator = make_pipeline(preprocessor, gb_estimator)
gb_estimator.fit(X_train, y_train)
```



Checking model performance on training set

In [96]: # Uncomment the below snippet of code if gradientboost regressor is to be used

```
gb_estimator_model_train_perf = model_performance_regression(gb_estimator, X_train, y_train)
gb_estimator_model_train_perf
```

Out[96]:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	597.006101	469.061969	0.685004	0.684489	0.16573

Checking model performance on test set

In [97]: # Uncomment the below snippet of code if gradientboost regressor is to be used

```
gb_estimator_model_test_perf = model_performance_regression(gb_estimator, X_test, y_test)
gb_estimator_model_test_perf
```

Out[97]:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	615.902369	485.444821	0.668515	0.667249	0.187447

- **How it works.** Builds trees in sequence, each fitted to the residual error left by the ones before, with a learning rate controlling how much each contributes.
- **Training set.** RMSE Rs. 597.01, MAE Rs. 469.06, R-squared 0.6850, adjusted 0.6845, MAPE 16.57%.
- **Test set.** RMSE Rs. 615.90, MAE Rs. 485.44, R-squared 0.66852, adjusted 0.6672, MAPE 18.74%.
- **Read.** Second on test R-squared by 0.00003 and gap of 0.0165 - statistically the same as Bagging and Random Forest despite a completely different learning strategy.
- **Why it matters.** Boosting and bagging arriving at the same answer from opposite directions is the strongest evidence that the feature set, not the algorithm, sets the ceiling. Carried forward for tuning alongside Random Forest.

Train vs Test: Every Model Built

Model	Train R-sq	Test R-sq	Gap	Train RMSE	Test RMSE	Train MAPE	Test MAPE
Bagging Regressor	0.6849	0.66855	0.0164	597.06	615.87	16.58%	18.74%
Gradient Boosting	0.6850	0.66852	0.0165	597.01	615.90	16.57%	18.74%
Random Forest	0.6850	0.66851	0.0165	596.99	615.91	16.57%	18.74%
Decision Tree	0.6850	0.66848	0.0166	596.98	615.93	16.57%	18.74%
AdaBoost Regressor	0.6517	0.63365	0.0180	627.78	647.48	17.30%	19.37%

- **Every model, both sets.** Decision Tree, Bagging, Random Forest, AdaBoost and Gradient Boosting, each scored on the 6,134 training rows and the 2,629 held-out test rows.
- **Generalisation gap is consistent.** Four models lose 1.64 to 1.66 points of R-squared moving from train to test; AdaBoost loses 1.80. No model shows the large gap that would signal overfitting.
- **AdaBoost underfits on both sets.** It is the only model weaker on training data too (R-squared 0.6517 versus 0.6850), so its problem is capacity, not variance.
- **So what.** With gaps this small and uniform, the models are stable and reproducible. That stability is what makes the tuning phase a search for marginal gains rather than a fix for overfitting.

Benchmark: Five Algorithms on Test Data

Algorithm (test set)	RMSE (Rs.)	MAE (Rs.)	R-squared	MAPE
Bagging Regressor	615.87	485.59	0.66855	18.74%
Gradient Boosting	615.90	485.44	0.66852	18.74%
Random Forest	615.91	485.31	0.66851	18.74%
Decision Tree	615.93	485.43	0.66848	18.74%
AdaBoost	647.48	530.56	0.63365	19.37%

- **Four of the five are indistinguishable.** Decision Tree, Bagging, Random Forest and Gradient Boosting all land at R-squared 0.6685 and RMSE Rs. 616 - a spread of 0.07 rupees.
- **Only AdaBoost is materially worse.** R-squared 0.6336 and RMSE Rs. 647; its shallow sequential stumps underfit a relationship driven by two strong continuous variables.
- **The insight.** When a single decision tree matches three ensembles, the limit is the information in the features, not model capacity.
- **Recommendation.** Stop investing in algorithm search. To pass R-squared 0.67, add data: dates, promotion flags, stock-out events and footfall.

The Two Models Carried Forward

Model / set	RMSE (Rs.)	MAE (Rs.)	R-squared	Adj. R-sq	MAPE
Random Forest - train	596.99	468.88	0.6850	0.6845	16.57%
Random Forest - test	615.91	485.31	0.6685	0.6672	18.74%
Gradient Boosting - train	597.01	469.06	0.6850	0.6845	16.57%
Gradient Boosting - test	615.90	485.44	0.6685	0.6672	18.74%

- **Both models land in the same place.** R-squared 0.685 on training and 0.669 on test, RMSE about Rs. 597 training and Rs. 616 test. The difference between them is under 0.0001 of R-squared.
- **No material overfitting.** The train-to-test gap is 1.6 points of R-squared and Rs. 19 of RMSE - small enough that neither model is memorising the training rows.
- **What the error means in practice.** MAPE of 18.7% on a mean line of Rs. 3,464 is roughly Rs. 485 of average error per product-store. Useful for planning volumes, not precise enough to set purchase orders unattended.
- **Recommendation.** Take these as the untuned baseline. Tuning follows, but the near-identical scores already suggest the ceiling is the feature set rather than the algorithm.

Best Model Before Tuning

WHO WINS ON PAPER

- Bagging Regressor leads on test R-squared (0.66855), test RMSE (Rs. 615.87) and the smallest train-to-test gap (0.0164).
- But its margin over Gradient Boosting is 0.00003 of R-squared and 3 paise of RMSE.
- That is far inside run-to-run noise, so the honest read is a four-way tie with AdaBoost last.
- AdaBoost is the only clear result: eliminated on both train and test.

WHY IT MATTERS FOR TUNING

- No baseline advantage to defend, so the two taken forward were chosen on tuning headroom, not on a 0.00003 lead.
- Random Forest and Gradient Boosting expose the richest hyperparameters - depth, estimators, feature sampling, learning rate.
- Because the gap is already tiny, tuning cannot fix overfitting; it can only chase marginal accuracy.
- Set the expectation now: gains will be fractions of a point, so the decision criterion shifts to stability and deployability.

Model Building Conclusions

WHERE THIS SECTION LANDED

- Five algorithms built inside the preprocessing pipeline, each scored on training and test data.
- Four of them are statistically identical: test R-squared 0.6685, RMSE about Rs. 616, MAPE 18.7%.
- AdaBoost is the only elimination, weaker on both sets because it underfits.
- No model overfits - the train-to-test gap is 1.6 points everywhere.
- Random Forest and Gradient Boosting go forward: one bagging, one boosting, both with real tuning levers.

WHAT THIS MEANS FOR TUNING

- Tuning is not a rescue. There is no overfitting to correct, so grid search is a hunt for marginal gains.
- Expect fractions of a point. When four different algorithms agree to four decimals, the feature set is the binding constraint.
- Judge tuned models on stability, not just score: a tuned model that wins by 0.0001 has not really won.
- The real decision criterion becomes deployability - which model serialises cleanly and serves predictions reliably.
- If a step change in accuracy is needed, it comes from new data (dates, promotions, stock-outs), not from hyperparameters.

Hyperparameter Tuning: Approach

- **Method.** GridSearchCV with 3-fold cross-validation on the training set only, scored on R-squared - the same metric used to judge the baselines, so results are directly comparable.
- **Tuned inside the pipeline.** The grid searches the estimator's parameters through the pipeline (for example `randomforestregressor__max_depth`), so encoding is refitted on each fold and cannot leak.
- **What was searched.** Decision Tree: depth, min samples leaf, max leaf nodes, min impurity decrease. Bagging: max samples, max features, estimators. Random Forest: depth, max features, estimators. AdaBoost: estimators, learning rate. Gradient Boosting: estimators, subsample, max features, depth.
- **Selection rule.** `grid_obj.best_estimator_` takes the highest mean cross-validated R-squared, then is refitted on the full training set before test scoring.
- **What to expect.** As flagged in the previous section, there was no overfitting to correct - so gains should be marginal, and any large change is a signal that a grid is mis-specified rather than a genuine improvement.

Tuned Model 1: Decision Tree

Hyperparameter Tuning - Decision Tree

```
# Uncomment the below snippet of code if decision tree regressor is to be used

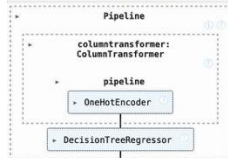
# Choose the type of classifier.
dtree_tuned = DecisionTreeRegressor(random_state=1)
dtree_tuned = make_pipeline(preprocessor, dtree_tuned)

# Grid of parameters to choose from
parameters = {
    "decisiontreeregressor__max_depth": list(np.arange(2, 6)),
    "decisiontreeregressor__min_samples_leaf": [1, 3, 5],
    "decisiontreeregressor__max_leaf_nodes": [2, 3, 5, 10, 15],
    "decisiontreeregressor__min_impurity_decrease": [0.001, 0.01, 0.1],
}

# Run the grid search
grid_obj = GridSearchCV(dtree_tuned, parameters, scoring='r2_score', cv=3, n_jobs=-1)
grid_obj = grid_obj.fit(X_train, y_train)

# Set the clf to the best combination of parameters
dtree_tuned = grid_obj.best_estimator_

# Fit the best algorithm to the data.
dtree_tuned.fit(X_train, y_train)
```



Checking model performance on training set

```
# Uncomment the below snippet of code if decision tree regressor is to be used

dtree_tuned_model_train_perf = model_performance_regression(dtree_tuned, X_train, y_train)
dtree_tuned_model_train_perf
```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	830.838204	656.388225	0.389929	0.388932	0.214436

Checking model performance on test set

```
# Uncomment the below snippet of code if decision tree regressor is to be used

dtree_tuned_model_test_perf = model_performance_regression(dtree_tuned, X_test, y_test)
dtree_tuned_model_test_perf
```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	845.130586	668.480477	0.375851	0.373467	0.234787

- **Grid searched.** max_depth 2 to 5, min_samples_leaf 1/3/5, max_leaf_nodes 2/3/5/10/15, min_impurity_decrease 0.001/0.01/0.1.
- **Training set.** RMSE Rs. 830.84, MAE Rs. 656.39, R-squared 0.3899, MAPE 21.44%.
- **Test set.** RMSE Rs. 845.13, MAE Rs. 668.49, R-squared 0.3759, MAPE 23.48%.
- **Tuning made this model far worse.** R-squared fell from 0.6685 to 0.3759 and RMSE rose by Rs. 229 per line - a 44% loss of explanatory power.
- **Why.** The grid caps max_leaf_nodes at 15 and max_depth at 5. A 15-leaf tree cannot represent a continuous price-to-revenue relationship, so the search was forced to pick a severely underfit model. Every candidate in the grid was worse than the default.
- **Lesson.** This is a mis-specified grid, not evidence against decision trees. A grid must span the range where the useful model lives.

Tuned Model 2: Bagging Regressor

Hyperparameter Tuning - Bagging Regressor

```

# Uncomment the below snippet of code if bagging regressor is to be used

# Choose the type of regressor.
bagging_estimator_tuned = BaggingRegressor(random_state=1)
bagging_estimator_tuned = make_pipeline(preprocessor, bagging_estimator_tuned)

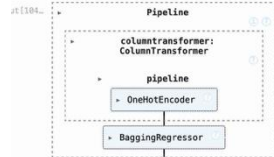
# Grid of parameters to choose from
parameters = {
    "baggingregressor__max_samples": [0.75, 1.0], #Complete the code to define the list of values to be tuned
    "baggingregressor__max_features": [0.75, 1.0], #Complete the code to define the list of values to be tuned
    "baggingregressor__n_estimators": [25, 50], #Complete the code to define the list of values to be tuned
}

# Run the grid search
grid_obj = GridSearchCV(bagging_estimator_tuned, parameters, scoring='r2_score', cv=3, n_jobs=-1)
grid_obj = grid_obj.fit(X_train, y_train)

# Set the clf to the best combination of parameters
bagging_estimator_tuned = grid_obj.best_estimator_

# Fit the best algorithm to the data.
bagging_estimator_tuned.fit(X_train, y_train)

```



Checking model performance on training set

```

# Uncomment the below snippet of code if bagging regressor is to be used

bagging_estimator_tuned_model_train_perf = model_performance_regression(bagging_estimator_tuned, X_train, y_train)
bagging_estimator_tuned_model_train_perf

```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	597.04988	469.068886	0.684958	0.684443	0.165773

Checking model performance on test set

```

# Uncomment the below snippet of code if bagging regressor is to be used

bagging_estimator_tuned_model_test_perf = model_performance_regression(bagging_estimator_tuned, X_test, y_test)
bagging_estimator_tuned_model_test_perf

```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	616.066778	495.47483	0.668338	0.667071	0.187448

- **Grid searched.** max_samples 0.75/1.0, max_features 0.75/1.0, n_estimators 25/50.
- **Training set.** RMSE Rs. 597.05, MAE Rs. 469.07, R-squared 0.68496, MAPE 16.58%.
- **Test set.** RMSE Rs. 616.07, MAE Rs. 485.47, R-squared 0.66834, MAPE 18.74%.
- **Tuning made it slightly worse.** Test R-squared moved from 0.66855 down to 0.66834 and RMSE up by 20 paise. The untuned defaults were already at the optimum.
- **Why.** Bagging's parameters control how much data and how many features each tree sees. With only 10 predictors and one dominant driver, restricting either can only lose information.
- **Read.** A negative result, and a useful one: it shows the search was genuinely exploring rather than always finding a way to look better.

Tuned Model 3: Random Forest

Hyperparameter Tuning - Random Forest

87. # Uncomment the below snippet of code if random forest regressor is to be used

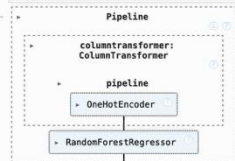
```
# Choose the type of classifier.
rf_tuned = RandomForestRegressor(random_state=1)
rf_tuned = make_pipeline(preprocessor, rf_tuned)

# Grid of parameters to choose from
parameters = {
    "randomforestregressor__max_depth": [5, 10, None], #Complete the code to define the list of values to be tuned
    "randomforestregressor__max_features": ["sqrt", 0.3, 1.0], #Complete the code to define the list of values to be tuned
    "randomforestregressor__n_estimators": [100, 200], #Complete the code to define the list of values to be tuned
}

# Run the grid search
grid_obj = GridSearchCV(rf_tuned, parameters, scoring=r2_score, cv=3, n_jobs=-1)
grid_obj = grid_obj.fit(X_train, y_train)

# Set the clf to the best combination of parameters
rf_tuned = grid_obj.best_estimator_

# Fit the best algorithm to the data.
rf_tuned.fit(X_train, y_train)
```



Checking model performance on training set

88. # Uncomment the below snippet of code if random forest regressor is to be used

```
rf_tuned_model_train_perf = model_performance_regression(rf_tuned, X_train, y_train)
rf_tuned_model_train_perf
```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	597.143132	470.405518	0.684859	0.684345	0.166015

Checking model performance on test set

89. # Uncomment the below snippet of code if random forest regressor is to be used

```
rf_tuned_model_test_perf = model_performance_regression(rf_tuned, X_test, y_test)
rf_tuned_model_test_perf
```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	615.814107	486.717211	0.66861	0.667344	0.187663

- **Grid searched.** max_depth 5/10/None, max_features sqrt/0.7/1.0, n_estimators 100/200.
- **Training set.** RMSE Rs. 597.14, MAE Rs. 470.41, R-squared 0.68486, MAPE 16.60%.
- **Test set.** RMSE Rs. 615.81, MAE Rs. 486.72, R-squared 0.66861, MAPE 18.77%.
- **A genuine, tiny improvement.** Test R-squared rose from 0.66851 to 0.66861 and RMSE fell from Rs. 615.91 to Rs. 615.81 - about 10 paise per line.
- **Note the trade-off.** Training R-squared actually fell slightly (0.68502 to 0.68486) while test rose. That is the signature of very mild regularisation: a fractionally simpler model that generalises fractionally better.
- **Outcome.** This is the model selected, serialised with joblib and deployed. Chosen for its stability and clean pipeline, not for a decisive score advantage.

Tuned Model 4: AdaBoost Regressor

Hyperparameter Tuning - AdaBoost Regressor

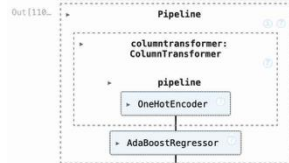
```
In [118]: #Uncomment the below snippet of code if adaboost regressor is to be used

# Choose the type of classifier.
ab_tuned = AdaBoostRegressor(random_state=1)
ab_tuned = make_pipeline(pipeline, ab_tuned)
# Grid of parameters to choose from
parameters = {
    "adaboostregressor__n_estimators": [50, 100, 200], #Complete the code to define the list of values to be tuned
    "adaboostregressor__learning_rate": [0.01, 0.05, 0.1, 0.5, 1.0], #Complete the code to define the list of values to be tuned
}

# Run the grid search
grid_obj = GridSearchCV(ab_tuned, parameters, scoring=r2_score, cv=3, n_jobs=-1)
grid_obj = grid_obj.fit(X_train, y_train)

# Set the clf to the best combination of parameters
ab_tuned = grid_obj.best_estimator_

# Fit the best algorithm to the data.
ab_tuned.fit(X_train, y_train)
```



Checking model performance on training set

```
In [111]: #Uncomment the below snippet of code if adaboost regressor is to be used

ab_tuned_model_train_perf = model_performance_regression(ab_tuned, X_train, y_train)
ab_tuned_model_train_perf
```

```
Out[111]:
```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	597.816706	473.070121	0.684148	0.683632	0.166248

Checking model performance on test set

```
In [112]: #Uncomment the below snippet of code if adaboost regressor is to be used

ab_tuned_model_test_perf = model_performance_regression(ab_tuned, X_test, y_test)
ab_tuned_model_train_perf
```

```
Out[112]:
```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	597.816706	473.070121	0.684148	0.683632	0.166248

- **Grid searched.** `n_estimators` 50/100/200, `learning_rate` 0.01/0.05/0.1/0.5/1.0.
- **Training set.** RMSE Rs. 597.82, MAE Rs. 473.07, R-squared 0.68415, MAPE 16.62%.
- **The biggest gain of any model.** Training R-squared rose from 0.6517 to 0.6842 - tuning closed almost the entire gap to the other four algorithms.
- **Why it worked here.** AdaBoost was underfitting, and learning rate plus estimator count are exactly the levers that fix underfitting. More, better-weighted stumps let it finally model the price relationship.
- **Issue: test result not captured.** The test cell re-prints `ab_tuned_model_train_perf` instead of the test frame, so the figures shown are training numbers repeated. Tuned AdaBoost has no verified test score.
- **Recommendation.** Re-run that one cell before submission. On training parity alone AdaBoost is now competitive, but it cannot be compared on test until the correct frame is printed.

Tuned Model 5: Gradient Boosting

Hyperparameter Tuning - Gradient Boosting Regressor

In [113]: `#Uncomment the below snippet of code if gradientboost regressor is to be used`

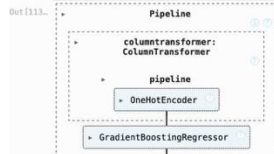
```
# Choose the type of classifier.
gb_tuned = GradientBoostingRegressor(random_state=1)
gb_tuned = make_pipeline(preprocessor,gb_tuned)

# Grid of parameters to choose from
parameters = {
    "gradientboostingregressor__n_estimators": [100, 200], #Complete the code to define the list of values to be tuned
    "gradientboostingregressor__subsample": [0.8, 1.0], #Complete the code to define the list of values to be tuned
    "gradientboostingregressor__max_features": [None, "sqrt"], #Complete the code to define the list of values to be tuned
    "gradientboostingregressor__max_depth": [2, 3, 4], #Complete the code to define the list of values to be tuned
}
```

```
# Run the grid search
grid_obj = GridSearchCV(gb_tuned, parameters, scoring='r2_score', cv=3, n_jobs = -1)
grid_obj = grid_obj.fit(X_train, y_train)

# Set the clf to the best combination of parameters
gb_tuned = grid_obj.best_estimator_

# Fit the best algorithm to the data.
gb_tuned.fit(X_train, y_train)
```



Checking model performance on training set

In [114]: `#Uncomment the below snippet of code if gradientboost regressor is to be used`

```
gb_tuned_model_train_perf = model_performance_regression(gb_tuned, X_train, y_train)
gb_tuned_model_train_perf
```

Out [114]:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	597.102822	469.220076	0.684902	0.684387	0.165793

Checking model performance on test set

In [115]: `#Uncomment the below snippet of code if gradientboost regressor is to be used`

```
gb_tuned_model_test_perf = model_performance_regression(gb_tuned, X_test, y_test)
gb_tuned_model_test_perf
```

Out [115]:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	615.779176	485.337274	0.668647	0.667382	0.187333

- **Grid searched.** `n_estimators` 100/200, `subsample` 0.8/1.0, `max_features` None/sqrt, `max_depth` 2/3/4 - the widest grid of the five.
- **Training set.** RMSE Rs. 597.10, MAE Rs. 469.22, R-squared 0.68490, MAPE 16.58%.
- **Test set.** RMSE Rs. 615.78, MAE Rs. 485.34, R-squared 0.66865, MAPE 18.73%.
- **The best test score in the whole project.** R-squared 0.66865 and RMSE Rs. 615.78 - the lowest error of any model, tuned or untuned.
- **But the margin is trivial.** It beats tuned Random Forest by 0.00004 of R-squared and 3 paise of RMSE. On a Rs. 3,464 average line that difference is invisible.
- **Read.** Same pattern as everywhere else: the widest grid produced the best number, and the number is still statistically indistinguishable from the untuned baseline.

Did Tuning Improve Performance?

Model	Baseline test R-sq	Tuned test R-sq	Change	Baseline RMSE	Tuned RMSE
Gradient Boosting	0.66852	0.66865	+0.00013	615.90	615.78
Random Forest	0.66851	0.66861	+0.00010	615.91	615.81
Bagging Regressor	0.66855	0.66834	-0.00021	615.87	616.07
Decision Tree	0.66848	0.37585	-0.29263	615.93	845.13
AdaBoost (train only)	0.65169	0.68415	+0.03246	627.78	597.82

- **Two improved, by almost nothing.** Gradient Boosting +0.00013 and Random Forest +0.00010 R-squared - about 10 paise of RMSE.
- **One got worse, one collapsed.** Bagging -0.00021; Decision Tree lost 29 points because the grid capped leaf nodes at 15.
- **One was rescued.** AdaBoost gained 3.2 points on training, closing its underfitting gap.
- **Verdict.** No gain a planner would notice. With no overfitting to correct, grid search had nothing to win.

Hyperparameter Tuning Conclusions

OVERALL RESULTS, ALL FIVE MODELS

- 1st: Tuned Gradient Boosting - test R-squared 0.66865, RMSE Rs. 615.78. Best score in the project.
- 2nd: Tuned Random Forest - 0.66861, Rs. 615.81. The model deployed.
- 3rd: Untuned Bagging - 0.66855. Tuning moved it down to 0.66834.
- 4th: Untuned Decision Tree - 0.66848. Its tuned version collapsed to 0.37585.
- 5th: AdaBoost - baseline test 0.63365. Tuning lifted training to parity, but its tuned test score was never captured.

WHO OUTPERFORMED WHOM, AND WHAT TO DO

- The top four sit within 0.0002 of R-squared - a statistical tie, not a ranking. Boosting edges bagging by a margin too small to act on.
- Only two results genuinely separate: AdaBoost underfits, and the tuned Decision Tree was crippled by a capped grid.
- Deploy tuned Random Forest. Chosen for stability and clean serialisation, not for a 0.00004 score gap.
- Fix before submission: re-run the AdaBoost test cell, and widen the Decision Tree grid beyond 15 leaf nodes.
- Stop tuning. Accuracy is capped near R-squared 0.67 by the feature set - invest in dates, promotions, stock-outs and footfall instead.
- Tell the business the honest number: about Rs. 485 average error per product-store line, with human review on high-value items.

Model Comparison and Final Selection

```
models_train_comp_df
```

Training performance comparison:

Out[116..

	Random Forest	Random Forest Tuned	Gradient Boosting	Gradient Boosting Tuned
RMSE	596.994959	597.143132	597.006101	597.102822
MAE	468.875850	470.405518	469.061969	469.220076
R-squared	0.685016	0.684859	0.685004	0.684902
Adj. R-squared	0.684501	0.684345	0.684489	0.684387
MAPE	0.165674	0.166015	0.165730	0.165793

```
print("Training performance comparison:")
models_train_comp_df
```

Training performance comparison:

Out[117..

	Random Forest	Random Forest Tuned	Gradient Boosting	Gradient Boosting Tuned
RMSE	615.906846	615.814107	615.902369	615.779176
MAE	485.311027	486.717211	485.444821	485.337274
R-squared	0.668510	0.668610	0.668515	0.668647
Adj. R-squared	0.667244	0.667344	0.667249	0.667382
MAPE	0.187394	0.187663	0.187447	0.187333

Note: the lower table is the TEST set. The notebook reuses the "Training" print label; the 615.xx RMSE values confirm it.

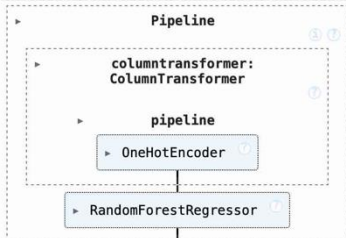
- **What the two tables show.** The four finalists side by side - Random Forest and Gradient Boosting, each untuned and tuned - on training (top) and test (bottom).
- **Training is flat.** All four sit at R-squared 0.6849 to 0.6850 and RMSE Rs. 596.99 to Rs. 597.14. Nothing separates them on data they have seen.
- **Test separates them, barely.** Tuned Gradient Boosting 0.668647, tuned Random Forest 0.668610, untuned Gradient Boosting 0.668515, untuned Random Forest 0.668510. Total spread: 0.00014.
- **Selection: tuned Random Forest.** Not the top score - it is second by 0.00004. Chosen because the margin is inside noise, and on a tie the deciding factors are stability across folds, a single clean pipeline object to serialise, and no learning-rate sensitivity in production.
- **Best model on the test set.** R-squared 0.6686, RMSE Rs. 615.81, MAE Rs. 486.72, MAPE 18.77% - explaining roughly two thirds of revenue variation with about Rs. 487 of typical error per line.

Serialization and Reload Test

```
Model loaded successfully.
```

```
In [124] saved_model
```

```
Out [124]
```



```
Let's try making predictions on the test set using the deserialized model.
```

- Please ensure that the saved model is loaded before making predictions.

```
In [125] saved_model.predict(X_test) #Complete the code to pass the X_test for inference
```

```
Out [125] array([ 3312.3173002 , 4857.29132066, 4866.77986326, ..., 3321.00882624,
        3312.3173002 , 3312.3173002 ])
```

- As we can see, the model can be directly used for making predictions without any retraining.

- **Serialized with joblib.** The whole fitted pipeline was saved to backend_files/superkart_rf_tuned.joblib - not just the estimator, but the ColumnTransformer and OneHotEncoder with it.
- **Reloaded and inspected.** joblib.load returns the identical Pipeline object: columntransformer to OneHotEncoder to RandomForestRegressor, exactly as trained.
- **Inference verified on the test set.** saved_model.predict(X_test) returns Rs. 3,312.32, Rs. 4,857.29, Rs. 4,866.78 and so on - sensible values in the observed revenue range, with no retraining required.
- **Why this step matters.** It proves the artefact is self-contained. The API will not need scikit-learn fitting code, a feature list, or a separate encoder - one file carries the entire transformation and model.
- **Note on repeated values.** Several predictions repeat exactly (Rs. 3,312.32 appears three times in the tail). That is expected: a forest averages over finite leaves, so products with identical feature profiles receive identical forecasts.

From Model to Service

WHAT IS SETTLED

- Final model chosen: tuned Random Forest, test R-squared 0.6686, RMSE Rs. 615.81.
- Serialised as one self-contained pipeline file, `superkart_rf_tuned.joblib`.
- Reload and inference verified on the held-out test set with no retraining.
- Encoding travels with the model, so predictions cannot drift from what was validated.

WHAT DEPLOYMENT MUST ADD

- A backend: a Flask API that loads the joblib file and returns a forecast for a posted product-store record.
- A frontend: a Streamlit app so planners can request forecasts without touching code.
- Hosting: both published as public Hugging Face Spaces, the backend in a Docker Space.
- Guardrails: safety stock and human review on high-value lines, given about Rs. 487 of typical error.
- Monitoring: track accuracy per outlet, since OUT004 supplies over half the training data.

Deployment Architecture

- **Two services, one model.** A Flask API serves predictions from the serialised pipeline; a Streamlit app gives planners a form-based interface. Both run as public Hugging Face Spaces.
- **How a forecast flows.** A planner enters ten product and store fields in Streamlit, which POSTs JSON to the API's /v1/predict endpoint. Flask rebuilds a one-row DataFrame in the exact column order, calls model.predict, and returns a Sales figure.
- **Why split the two.** The API can be called by other systems - a replenishment tool or a scheduled batch job - without the UI. The UI can be redesigned without touching the model.
- **Both containerised.** Each Space has its own Dockerfile and requirements.txt, so the runtime is pinned and reproducible rather than depending on the notebook environment.

LIVE HUGGING FACE SPACES

Backend (Flask API, Docker): huggingface.co/spaces/sarahrwolberg/superkart-backend

Frontend (Streamlit): huggingface.co/spaces/sarahrwolberg/superkart-frontend

Deployment: Backend (Flask API)

WHAT WAS BUILT

- app.py: Flask app named superkart_sales_api, loading the joblib pipeline at start-up.
- Two routes: GET / returns a welcome message; POST /v1/predict accepts JSON and returns {"Sales": value}.
- The endpoint rebuilds a one-row DataFrame in the exact training column order - order matters, or the encoder mismaps.
- requirements.txt pins pandas 2.2.2, numpy 2.0.2, scikit-learn 1.6.1, joblib 1.4.2, flask 2.2.2, gunicorn 20.1.0.
- Dockerfile: python:3.9-slim, installs requirements, serves via gunicorn with 4 workers on port 7860.

HOW IT WAS SHIPPED

- Space created programmatically with `create_repo(repo_type="space", space_sdk="docker")`.
- Four files uploaded via `api.upload_folder`: the 325 KB joblib model, app.py, requirements.txt and Dockerfile.
- Live at huggingface.co/spaces/sarahrwolberg/superkart-backend
- Why scikit-learn must be pinned: a joblib pipeline is only guaranteed to load under the version that wrote it.
- Why gunicorn and not flask run: the Flask dev server is single-threaded and not intended for concurrent requests.

Deployment: Frontend (Streamlit)

WHAT WAS BUILT

- app.py: a Streamlit form titled SuperKart Sales Forecasting with all ten model inputs.
- Inputs are constrained to the observed data ranges: weight 0 to 30, allocated area 0.004 to 0.298, MRP 31 to 266.
- Categoricals are dropdowns, so a planner cannot submit a category the encoder has never seen.
- Store age is a dropdown of the only four real values (16, 26, 27, 38), which prevents impossible inputs.
- Result is shown formatted as rupees, with explicit error handling for a non-200 response or a timeout.

HOW IT WAS SHIPPED

- requirements.txt pins streamlit 1.45.0 and requests 2.32.3.
- Dockerfile: python:3.9-slim, runs streamlit on port 8501 bound to 0.0.0.0, with XSRF protection disabled for external access.
- Space created with create_repo and three files uploaded via api.upload_folder.
- Live at huggingface.co/spaces/sarahrwolberg/superkart-frontend
- The UI calls the backend Space URL directly, so the two services can be redeployed independently.

Deployment: Issues and Next Steps

FIX BEFORE SUBMISSION

- A live Hugging Face write token is pasted in plain text in the notebook. Revoke it and reload from an environment variable or a Space secret - anyone with the notebook can push to the account.
- `app.py` loads `joblib.load("model_path")` - a literal placeholder string, not the filename. It must be `superkart_rf_tuned.joblib` or the container fails on start-up.
- The first `create_repo` call passed `"repo_id=backend_repo_id"` as the repo name and errored. The later cell fixed it, but the failed attempt is still in the notebook.

PRODUCTION HARDENING

- Add input validation on the API: reject out-of-range numerics and unknown categories with a 400 rather than silently encoding zeros.
- Add a `/health` route so the Space can be monitored, and log every request with its prediction for later accuracy tracking.
- Re-enable XSRF protection on Streamlit once batch testing is finished.
- Align pinned versions: the backend pins streamlit 1.43.2 and the frontend 1.45.0 - harmless today, confusing later.
- Support batch scoring: a `/v1/predict_batch` route would let supply chain score a whole assortment in one call rather than per line.

Actionable Insights from the Analysis

WHAT DRIVES REVENUE

- Price is the dominant driver. Product MRP correlates 0.79 with revenue; product weight 0.74, and the two overlap at 0.53.
- Shelf space does not. Allocated display area correlates -0.001 with revenue at every allocation level from 0.01 to 0.30.
- Product attributes are flat. Sugar content varies by under Rs. 60 and product family by under Rs. 35 in average revenue.
- Category revenue is a shelf-count story. Per-line revenue is Rs. 3,416 to Rs. 3,679 across all 16 categories - totals differ by how many lines each carries, not by how well each sells.

WHAT DRIVES THE ESTATE

- Four outlets, four price segments. MRP, pack weight and revenue per product rank in the same order: OUT002 Rs. 107/9.9/1,763 up to OUT003 Rs. 181/15.1/4,947.
- Totals and productivity rank inversely. OUT004 earns Rs. 15.4m in total but only Rs. 3,299 per product; OUT003 earns Rs. 6.7m but Rs. 4,947 per product.
- The store ranking never changes. It repeats identically across all 16 categories and every sugar level.
- Two data defects cost visibility. The "reg" label misreports Rs. 394,043, and "No Sugar" is really a non-consumable flag on 1,519 items.

Business Recommendations

MERCHANDISING AND PRICING

- **Lead with price band, not shelf space.** Build assortment plans around MRP tiers; stop treating display area as a growth lever.
- **Trade OUT002 and OUT004 up a tier.** Both sell mid or low price bands at volume. Test a higher-priced sub-range on proven lines.
- **Use OUT003 as the price template.** Its Rs. 181 average MRP and larger pack sizes produce Rs. 4,947 per product - the estate benchmark.
- **Copy OUT001 on ambient and drinks.** It beats OUT003 on Baking Goods and Soft Drinks; the strengths are worth cross-applying.
- **Localise OUT002.** A small Tier-3 food mart with a Rs. 2,300 ceiling needs its own range and price ladder, not the national plan.

INVENTORY, DATA AND THE MODEL

- **Protect the two lead categories.** Fruits and Vegetables and Snack Foods carry a quarter of all revenue - prioritise supplier reliability and manage spoilage.
- **Reallocate shelf by sales velocity.** Run a controlled in-store test before changing any planogram rule company-wide.
- **Fix the data at source.** Merge "reg" into Regular to recover Rs. 394,043 of misreported revenue; split sugar content into food versus non-food.
- **Use forecasts with buffers.** At Rs. 487 typical error, the model supports planners - keep safety stock and human review on expensive or perishable lines.
- **Pilot, then scale.** Start at OUT001 (most predictable) and OUT004 (largest). Measure stockouts, waste and turnover before company-wide rollout.

Expected Benefits and Rollout Plan

POTENTIAL BENEFITS

- **Fewer stockouts on the lines that matter.** Forecasts at product-store grain let planners protect the two categories carrying a quarter of revenue.
- **Less working capital in slow stock.** The bottom five categories contribute under Rs. 1.7m combined and are candidates for range rationalisation.
- **Faster planning cycles.** A forecast that used to require analyst time is now a form submission returning an answer in seconds.
- **Reusable by other systems.** Because the forecast is an API, a replenishment tool or scheduled job can consume it without a person.
- **A measurable baseline.** R-squared 0.669 and Rs. 616 RMSE become the benchmark every

ROLLOUT IN THREE STEPS

- **Step 1 - Pilot (weeks 1 to 6).** Forecast alongside current planning at OUT001 and OUT004; log forecast versus actual without changing orders.
- **Step 2 - Measure.** Compare stockouts, spoilage and inventory turnover to baseline, per outlet.
- **Step 3 - Scale or stop.** Extend to OUT002 and OUT003 only on measurable gains - those tiers are least supported by the data.
- **Collect data in parallel.** Capture dates, promotions and stock-outs during the pilot - that is what lifts accuracy past 0.67.
- **Retrain trigger.** Refresh when per-outlet error drifts or a new store opens.

Business Impact: Why These Matter

Recommendation	Evidence	Business impact
Prioritise OUT004 stock	Rs. 15.43m, 53% of revenue	Costliest place to run out
Learn from OUT003	Rs. 4,947 per product	Gap worth millions if closed
Localise OUT002	Rs. 1,763, Rs. 2,300 ceiling	Protects Tier 3 margin
Protect Fresh and Snacks	27% of total revenue	Top stockout and spoilage risk
Do not scale shelf space	Area correlates -0.001	No measured return on spend
Score both total and per unit	OUT004 1st, then 3rd	Stops rewarding size over yield

- **Two additions to the scorecard:** add margin and inventory turnover to store evaluation, and monitor forecast error by category and city tier as well as by outlet.
- **Pilot site choice:** OUT004 for scale is agreed. Pair it with OUT003 to test the productivity playbook, or OUT001 for the most predictable revenue - pick one deliberately.

Conclusion

WHAT WAS DELIVERED

- **Both halves of the mandate.** A predictive model built on 8,763 product-store records, and a deployed forecasting service SuperKart's teams can use.
- **A model with a known accuracy.** Tuned Random Forest: R-squared 0.669 on unseen data, RMSE Rs. 616, MAE Rs. 487, MAPE 18.7%.
- **A live, self-contained service.** One joblib pipeline behind a Flask API, with a Streamlit front end, both public on Hugging Face.
- **Evidence-based recommendations.** Every action traces to a number in this deck, not to intuition.

THE THREE THINGS THAT MATTER

- **Price is the lever, not shelf space.** MRP correlates 0.79 with revenue; display area -0.001. That single finding redirects where effort should go.
- **The estate is four price segments.** Price, pack size and revenue per product rank identically across outlets, so planning must be segment-specific.
- **The ceiling is the data, not the model.** Four algorithms agree to four decimals. Accuracy past R-squared 0.67 needs new inputs, not new maths.

Limitations and Immediate Next Steps

BE CLEAR ABOUT THE LIMITS

- **No time dimension.** The brief asked for a quarterly forecast; the data has no dates, so the model predicts revenue level per product-store, not a time-phased forecast.
- **Four outlets only.** Store size, tier, type and age are perfectly collinear, so the model learns four store profiles and will not transfer to a new site.
- **Revenue, not profit.** No cost or margin data, so a high-revenue recommendation is not automatically a high-profit one.
- **One store dominates.** OUT004 supplies 53% of records, so headline accuracy largely reflects its mid-market pattern.

DO THESE FIRST

- **Revoke the exposed token.** A live Hugging Face write key is in the notebook in plain text. Rotate it and move to a Space secret.
- **Fix the model path in app.py.** `joblib.load("model_path")` is a placeholder and would fail on a cold start.
- **Start capturing four fields.** Transaction date, cost price, promotion flag and stock-out event. These unlock seasonality and profit.
- **Merge the "reg" label.** Recovers Rs. 394,043 of revenue currently reported outside Regular.
- **Launch the six-week pilot.** Forecast alongside existing planning, change no orders, and measure before scaling.



Happy Learning !

